

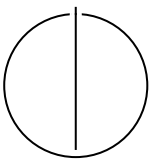
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Towards Criticality-based Value Prediction

Jakob Lison





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

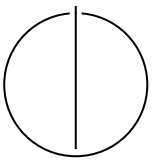
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Towards Criticality-based Value Prediction

Auf dem Weg zu einer kritikalitätsbasierten Wertvorhersage

Author:	Jakob Lison
Examiner:	Prof. Pramod Bhatotia
Supervisor:	Dr. David Schall
Submission Date:	17.08.2026



Acknowledgments

I would like to thank my supervisor, Dr. David Schall, for his guidance, detailed feedback, and support during the development of this thesis. I am particularly grateful for his help with the gem5 framework, the benchmark infrastructure, and the interpretation of the experimental results. I would also like to thank Prof. Pramod Bhatotia for examining this thesis and the Technical University of Munich for providing the academic framework for this work. Finally, I would like to thank my family and friends for their continued encouragement and support.

Abstract

Value prediction leverages value locality to predict instruction results before they are computed. By making these values available early, value prediction can break up true dependence chains, which is one of the biggest limitations of out-of-order execution in state-of-the-art processors. Modern value predictors try to achieve the highest speedup by increasing the coverage of the instructions they predict. Storing all these values requires significant space in the prediction table. However, because only a portion of the prediction table is usable for prediction, while the rest is required for training or is blocked by unpredictable instructions, the space is used inefficiently.

This thesis investigates a first step towards criticality-based value prediction. Instead of predicting as many instructions as possible, it focuses the available table capacity on load instructions that exhibit predictable behavior. To this end, we design and implement a two-level load-value predictor. A compact filter table handles the initial training phase and prevents unpredictable and insufficiently trained instructions from blocking space in the main prediction table and reducing efficiency.

The proposed predictor is implemented in the gem5 O3CPU model and evaluated against a conventional stride load-value predictor using 13 server and SPEC benchmarks. With a 256-entry filter table and a 512-entry main prediction table, the two-level predictor can achieve comparable average performance within the evaluated simulation to an 8192-entry stride value predictor. With an entry size of 19 bytes, the total table size drops from 156KB to 15KB, a reduction of 90.4%.

The results indicate that predictability-based filtering can improve prediction table capacity utilization. The proposed design does not yet use information about instruction criticality. It should therefore be understood as a first step towards criticality-based value prediction.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**

2 Background **5**
2.1 Motivation for Value Prediction 5
2.2 Functionality of a Value Predictor 5
2.3 Types of Value Predictors 6
2.4 State of the Art Value Predictors 7
2.5 gem5 8

3 Motivation **9**
3.1 Problems of Modern Value Predictors 9
3.2 Analysis of Stride Predictor 9
3.2.1 Training Time 10
3.2.2 Predictable Loads 11
3.2.3 Table Behavior 13
3.3 Conclusion and Goals 14

4 Design and Implementation **18**
4.1 Stride Predictor Base 18
4.2 Filter Design 19
4.2.1 Filter Concept 19
4.2.2 Entry of Filter 21
4.2.3 Table as Filter 21
4.3 Integration into Stride Predictor Implementation 22

5 Evaluation **24**
5.1 Methodology 24
5.2 Comparison of Different Filter Configurations 26
5.2.1 Size of Filter 26

5.2.2 Entry Threshold to Main Filter 26
5.3 Comparison to Stride Predictor and Different Sizes of Main Table 27
5.3.1 Prediction Table Efficiency 28
5.3.2 Handling Bursts of New Loads 32
5.3.3 Speedup 35
5.3.4 Coverage 35
5.4 Other Entry Thresholds 37

6 Related Work **39**

7 Summary and Conclusions **41**

8 Future Work **45**

Abbreviations **47**

List of Figures **48**

List of Tables **50**

Bibliography **51**

1 Introduction

Modern processor performance is increasingly bottlenecked by memory access latency, which severely limits instruction-level parallelism. When an instruction depends on a load value, it stalls until that data is available. This bottleneck is especially problematic during cache misses in the upper levels of the memory hierarchy, where latencies far exceed those of standard arithmetic operations.

While out-of-order execution partially bypasses this bottleneck by executing independent instructions out of program order while waiting for operands, its effectiveness depends entirely on the instruction window containing sufficient independent work. Once the window is empty of independent tasks, unresolved data dependencies halt further progress. A true data dependency creates a strict critical path when the producing instruction has high latency—a scenario typified by load instructions, whose execution time depends heavily on where the requested data resides in the memory hierarchy [HP17].

Value prediction bypasses this by predicting an instruction's result before the computation completes. This lets dependent instructions use the speculative value and proceed immediately. Crucially, value prediction does not eliminate the underlying dependency; it merely allows the pipeline to advance under the assumption that the guess is correct. If a misprediction occurs, the processor must detect the error and restore the speculative state. However, when accuracy is high enough, this mechanism breaks long dependency chains and unlocks new instruction-level parallelism, making it an excellent complement to traditional out-of-order engines [LWS96].

A key challenge in value prediction is balancing prediction coverage, prediction accuracy, hardware cost, and access latency. Boosting prediction coverage usually requires larger tracking tables and more complex logic. These changes increase storage requirements and may lengthen the predictor's critical access path. Therefore, a high-coverage predictor is counterproductive if the hardware footprint or latency overhead erases the performance gains.

State-of-the-art predictors seek performance gains by scaling coverage, relying on complex tracking schemes and often multiple tables to predict as many instructions as possible [PS14; Sez18; Gup+19; KHI19]. However, this approach carries a steep storage penalty. A single predictor entry often requires about 19 bytes; using these with a standard 8K-entry table totals roughly 160KB [PS14]. A footprint that rivals or exceeds

typical 32KB to 128KB L1 data caches [HP17].

Adding to this issue, storage utilization across these tables is highly uneven. Many slots are wasted on training instructions that never become predictable, or are consumed by entries stuck in training. Only a fraction of the table actively drives performance. In fact, our analysis in Chapter 3 shows that across various server and SPEC benchmarks, only about 21% of dynamic loads are predicted even once. The remaining loads are either completely unpredictable or tie up table capacity during training, never reaching the confidence threshold required for predictive viability.

These findings suggest that expanding table size is a brute-force approach to a structural problem. Instead, maximizing the efficiency of existing storage can yield equivalent performance with far fewer resources. The core premise of this thesis is to untangle the initial training phase from long-term prediction storage. By introducing a compact pre-filter table, we evaluate the early behavior of load instructions and advance only highly predictable ones to the main prediction table. This two-level organization prevents unpredictable loads from occupying the main table and reduces the number of entries wasted on instructions that never contribute to speedup.

Research Questions

This thesis addresses the following core research questions:

1. Can predictability-based pre-filtering measurably improve the storage efficiency of stride-based load-value prediction?

2. Is it possible for a two-level predictor to match the performance of a conventional stride predictor while using fewer total entries?

3. How sensitive are speedup, coverage, and table utilization to changes in filter size and entry thresholds?

4. Can criticality-related metrics serve as alternative entry criteria for the predictability-based filter?

The fourth research question is addressed only in an exploratory manner, because the main predictor design developed and evaluated in this thesis remains based on predictability rather than on criticality.

To address these questions, we begin by analyzing a standard stride predictor to quantify exactly how much capacity is lost to unpredictable and training loads. Using these insights, we establish clear design targets for a leaner predictor framework. We then introduce a two-level predictor architecture that uses a small pre-filter table to vet promising loads before they reach the main prediction array. This design reduces

pressure on the main table, thereby freeing up space for highly predictable loads. We implemented this architecture in the gem5 simulator by coupling the filter design with the stride predictor. To evaluate this implementation, we ran an extensive comparison of different filter configurations. Using these results, we compared the two-level predictor with the stride predictor across multiple factors for both server and SPEC workloads.

Although the long-term goal of this work is to investigate criticality-based value prediction, the predictor developed in this thesis does not yet use criticality information in its selection decisions. Instead, it uses early predictability to determine which load instructions to admit to the main prediction table. The proposed two-level predictor should therefore be understood as a first step toward a criticality-based design, providing the infrastructure for incorporating criticality information in future work.

Thesis Contributions

The primary contributions of this work include:

- Predictor Behavior Analysis: A detailed profiling of training and replacement inefficiencies in stride-based load-value predictors running server and SPEC workloads.
- Two-Level Architecture: A hardware design that decouples initial training from long-term prediction tracking using a compact pre-filter table.
- Simulator Implementation: A fully functional implementation of this two-level design in the gem5 O3CPU model, integrated with the baseline stride load-value predictor codebase.
- Trade-off Exploration: An analysis of the trade-off between storage savings and filter selectivity and their impact on performance to identify the most suitable configuration.
- Design Evaluation: An in-depth evaluation of storage behavior, coverage, and execution speed comparing the two-level design with the stride predictor.

Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 introduces the key concepts in value prediction and describes the gem5 simulation environment. Chapter 3 analyzes the stride predictor's behavior and derives the motivation for the proposed filtering mechanism. Chapter 4 presents the architecture of the two-level predictor and

its implementation in gem5. Chapter 5 evaluates storage requirements, prediction table behavior, and performance using the selected benchmarks. Chapter 6 discusses related work and situates the proposed mechanism within existing value-prediction techniques. Chapter 7 concludes the thesis, and Chapter 8 outlines directions for future work.

2 Background

2.1 Motivation for Value Prediction

In modern processors, the gap in processing speed has grown to the point that it is now one of the main limitations on overall performance. The main reason is that many instructions must wait for load results to become available before they can be processed. This can often take multiple orders of magnitude longer than other operations, delaying subsequent calculations.

One way to work around these long latencies is to use out-of-order scheduling. With out-of-order scheduling, instructions execute in any order between the issue and commit stages, if data dependencies are respected. This allows the processor to schedule other non-data-dependent instructions while waiting for a load result to return, so it does not have to stall. If there aren't enough independent instructions available, the processor stalls due to data dependencies.

True data dependencies occur when the result of one instruction is needed by a subsequent instruction. If producing the result takes a long time, the processor has to stall, wasting processing time. This is especially relevant for load results, since memory accesses can be time-consuming.

Value prediction can break these dependencies by predicting the value before its computation completes. This reduces stalling caused by long wait times and breaks up dependence chains, thereby increasing Instruction-Level Parallelism (ILP) and bypassing one of the biggest limitations of out-of-order execution in state-of-the-art processors. [HP17]

2.2 Functionality of a Value Predictor

A value predictor consists of three parts. The first part is a table that stores the Program Counter (PC) tag for each instruction we try to predict, as shown in 2.1. Additionally, we need to store the prediction values and other data required for predictions. The key column for the prediction process is confidence, which tracks how many consecutive times a prediction was correct and, therefore, the "confidence" we have in predicting this value. If a prediction is wrong once, this value is reset to zero to ensure high

PC Tag	Confidence	Value	...
...	...	...	...
110458	0	1	...
101242	25	0	...
103274	10	128	...
...	...	...	...

Figure 2.1: Example structure of a value predictor table. Each entry stores the instruction's PC tag, confidence counter, last value, and other information necessary for predicting the next value and information for the replacement algorithm.

accuracy.

The second part is the prediction scheme. The system starts predicting with this scheme once the confidence value exceeds a preset threshold, ensuring that the instruction follows the same pattern in subsequent executions. For value predictors, this value is often set relatively high because the cost of a misprediction is usually much greater than the number of cycles saved by a correct prediction. When prediction is initiated, multiple schemes can be used to target different value patterns. Both the storage table and the prediction scheme are located at the front-end of the pipeline because the goal is to make the predicted instruction result available as early as possible.

The final step is prediction verification, which occurs during the pipeline's commit stage. Here, the processor checks whether the predicted value matches the actual result and updates the confidence. In the case of a misprediction, we must roll back the pipeline and restart instruction execution. Additionally, the prediction value is updated, and a new training sequence begins. [LWS96]

2.3 Types of Value Predictors

Value predictors are classified into two main categories. The first category is computational predictors, which use operations to predict the next value based on the previous one. There are two main types of value predictors in this category. The first

and simplest type of value predictor is a constant-value prediction (also known as Last Value Prediction). To predict constant values, the predictor only needs to save the instruction's return value, along with the PC tag, and the confidence. While this allows for a very simple design, it also restricts predictions to only constant values and ignores other patterns that could otherwise be predicted. The other type, one level above a constant-value predictor, is a stride-value predictor. It can predict an additional group of instructions by storing a stride value, enabling it to predict instructions that change with a constant stride by adding the stride to the last instruction's result. Additionally, it must account for how many instances of the same instruction are currently in the pipeline to correctly calculate how many strides it needs to add to match the next value produced by this instruction. These are called in-flight instructions.

The other category of value predictors is context-based, which uses previously detected patterns and compares them with current patterns to predict values. This can be achieved either by using a history of past values to base the prediction on, the so-called Finite Context Method (FCM), or by using the branch history to provide context for the control flow. By employing these context-based prediction schemes, predictors can predict any repeating behavior that is neither strictly a stride nor a constant pattern, thereby substantially increasing coverage. For value predictors, coverage is the number of values predicted divided by the number of values they could have predicted. [SS97]

2.4 State of the Art Value Predictors

The current tactic for state-of-the-art predictors to improve performance is to increase their coverage by using complex prediction schemes and combining stride and context prediction to predict as many possible values as possible. For example, the Value Tagged Geometric History Length Predictor (VTAGE) [PS14] combines a base predictor with multiple tagged predictor tables, each indexed by different lengths of global branch history. By using these geometrically scaled history lengths, the predictor can capture both short-term and long-term context patterns. This allows VTAGE to achieve very high coverage and accuracy, but it also significantly increases overall complexity and storage requirements of the hardware. Enhanced VTAGE Enhanced Stride (EVES) [Sez18] takes this one step further by combining VTAGE with an enhanced stride predictor. To achieve even higher coverage, a predictor like the STEVES combines the EVES predictor with an FCM predictor [Gup+19]. In the direction of FCM is the H3VP that combines speculations from three individual predictors to predict values [KHI19]. Overall, we observe that state-of-the-art predictors combine multiple prediction methods to capture as many patterns as possible.

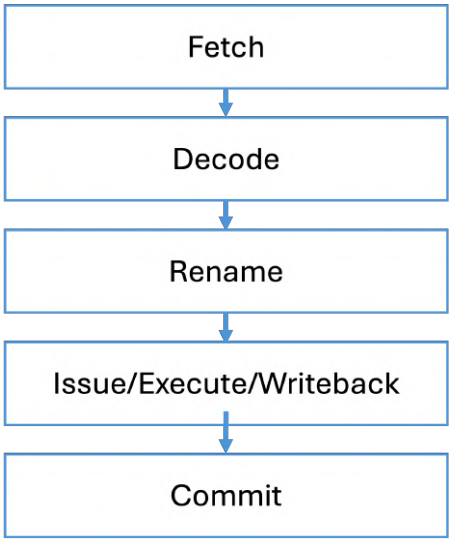


Figure 2.2: 5-stage pipeline structure of the gem5 O3CPU model

2.5 gem5

To evaluate the performance and resource trade-offs of these complex prediction schemes, physical hardware implementation is often impractical. Instead, architectural simulation models processor behavior and measures the impact of different predictor configurations under realistic workloads. In this thesis, the gem5 simulator is used to implement and analyze value predictors in a cycle-level environment that supports out-of-order execution. gem5 is an open-source, modular simulation platform that enables highly detailed modeling of modern out-of-order CPUs and memory subsystems [Low+]. The simulation framework used in this work is based on the gem5-fdp repository maintained by Dr. David Schall at the Technical University of Munich [Scha]. This repository extends the main gem5 codebase with a state-of-the-art front-end, including fetch-directed instruction prefetching and an enhanced branch predictor infrastructure. The value predictor implementation is part of the "value-pred" development branch of the gem5-fdp repository. The specific CPU model used in this work is the "O3CPU", an out-of-order CPU modeled after the Alpha 21264 [Pro]. In this model, the three pipeline stages, Issue, Execute, and Writeback, are combined into one stage. Together with the front-end stages Fetch, Decode, and Rename, and the Commit stage, the O3CPU provides a 5-stage pipeline, as depicted in Figure 2.2.

3 Motivation

3.1 Problems of Modern Value Predictors

As explained in Section 2.4, state-of-the-art value predictors try to increase coverage to improve performance. To achieve higher coverage, they require additional resources, as they must process and, in particular, store more values simultaneously [Ban+20]. In particular, the tables can easily become large, as each entry must store a tag, value, confidence, replacement-strategy data, and other information such as a stride or context. Even for the same predictor, larger tables improve the performance, as seen in Figure 3.1. The graph shows the average speedup of 13 benchmarks compared to a no-predictor simulation for different numbers of entries in the prediction table. We can see that with larger prediction tables, the predictor achieves a higher speedup.

Additionally, these numbers represent only the number of entries. If we calculate the size, a single entry can easily be 19 bytes [1]. Assuming a standard table size of 8K, the table already occupies 160KB [PS14]. This is quite large compared to an L1 data cache, which typically ranges from 32KB to 128KB [HP17]. These findings show that simply increasing the prediction-table size is not necessarily an efficient solution. Larger tables can store more instructions, but they also require additional hardware resources and may still be occupied by entries that are only being trained or never become predictable. We therefore investigate how the available table capacity is used and whether it can be managed more efficiently. The following section analyzes the behavior of a stride-based value predictor in more detail.

3.2 Analysis of Stride Predictor

To better understand how the value predictor behaves and how we can reduce our resource demand, we analyzed a stride load value predictor in gem5. We investigated how the predictor operates under different settings and analyzed the behavior of the overall predictor, loads, and the prediction table to draw conclusions. In all experiments, we used a load value predictor that predicts only load instructions. Because memory accesses can often take much longer than other instructions, especially when they miss

[1] 2-byte tag + 8-byte value + 8-byte stride + 6-bit confidence counter + 1 useful bit + 1 valid bit

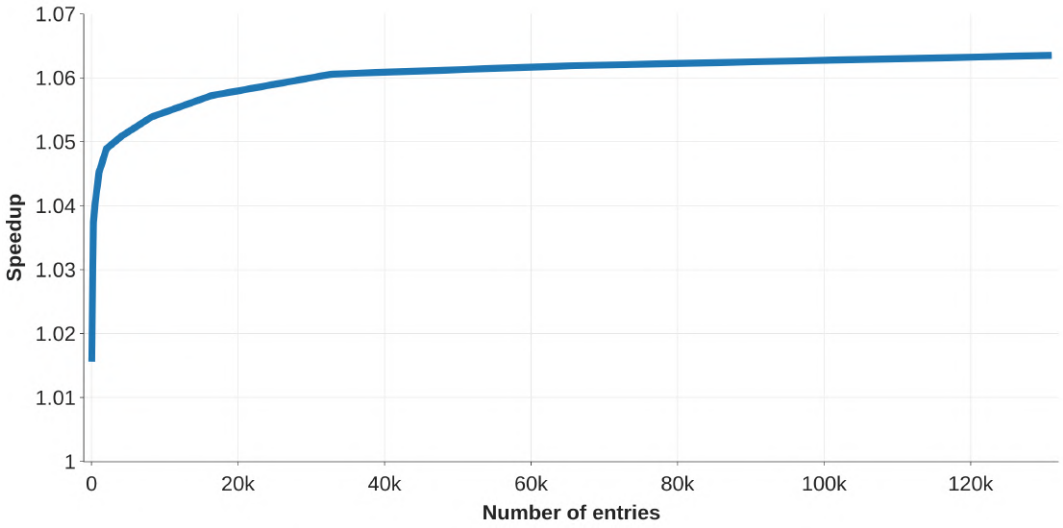


Figure 3.1: Average stride predictor speedup of 13 different benchmarks compared to the no predictor baseline, plotted over the number of prediction table entries.

the L1 cache, load instructions offer up to 10x the saving potential of other instructions, making it efficient to focus prediction only on loads [OAM18]. To test the predictor, we examine its behavior across 13 different workloads, as presented in Table 5.1. The complete specifications used in these simulations are the same as those described for the stride predictor in Section 5.1.

3.2.1 Training Time

A key factor in the predictor's behavior is the confidence threshold, which determines how long training for the loads takes and, therefore, the coverage and accuracy. Figure 3.2 shows the stride predictor speedup compared to a no-predictor baseline. We calculated the speedups for different benchmarks at confidence thresholds from 2 to 200 to assess their impact on performance. The bold dashed red line represents the average across all 13 benchmarks.

While the average already achieves a speedup at a confidence threshold of 6, some benchmarks still show slowdowns even up to a confidence threshold of 50. To ensure sufficient accuracy, modern predictors use high thresholds. For example, the value predictor used in the Apple M3 chip only consistently activates after 250 correct predictions [Kim+25]. Additionally, we observe that the "renaissance-chirper" benchmark

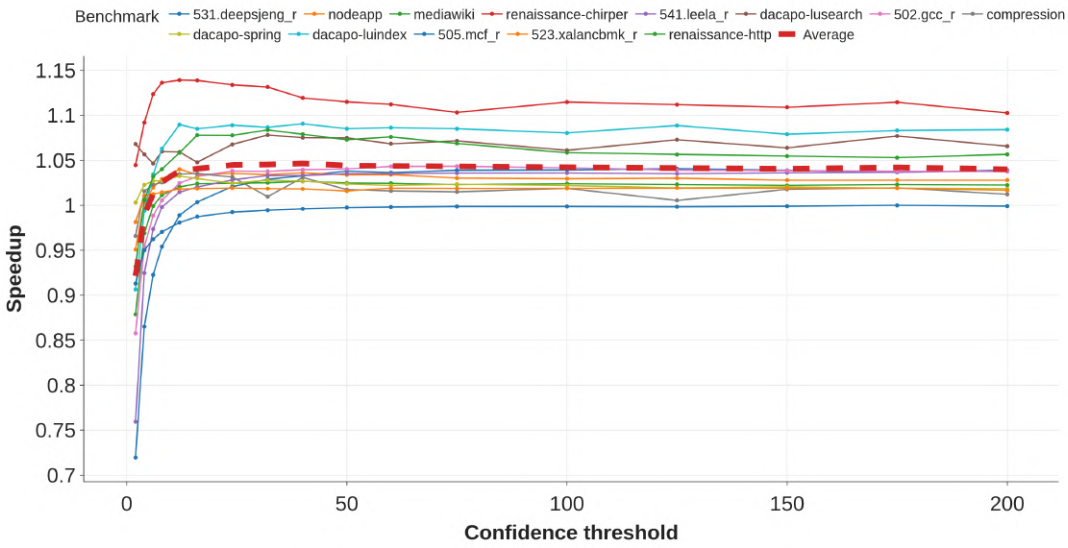


Figure 3.2: Stride predictor speedup across 13 different workloads compared to the no predictor baseline, plotted for confidence thresholds between 2 and 200.

shows a decrease in speedup at higher thresholds.

To ensure sufficient accuracy in our simulations and avoid losing coverage, we conducted all subsequent experiments with a confidence threshold of 50. Beyond accuracy, these high-confidence thresholds affect how long a value must be trained and, therefore, how long it takes to become useful for prediction. Higher thresholds, therefore, also waste more space in the prediction table, as more space is used for training instead of actual prediction.

3.2.2 Predictable Loads

To better understand how to optimize the space of the prediction table, we examine how many loads are predictable. Not all loads are predictable. However, the value predictor must still train on all loads to determine which are predictable, thereby wasting unnecessary entries. Figure 3.3 shows how many executed loads are predictable, displaying the percentage of all loads predicted at least once across the entire simulation for all benchmarks. These percentages range from 1.8% to 49.2%. Overall, the average is 21% of all loads, shown as the dotted red line.

Figure 3.4 shows the absolute number of executed loads and marks the prediction-table capacity with a red dotted line. In combination with Figure 3.3, the figure indicates that benchmarks with fewer executed loads generally achieve a higher fraction of

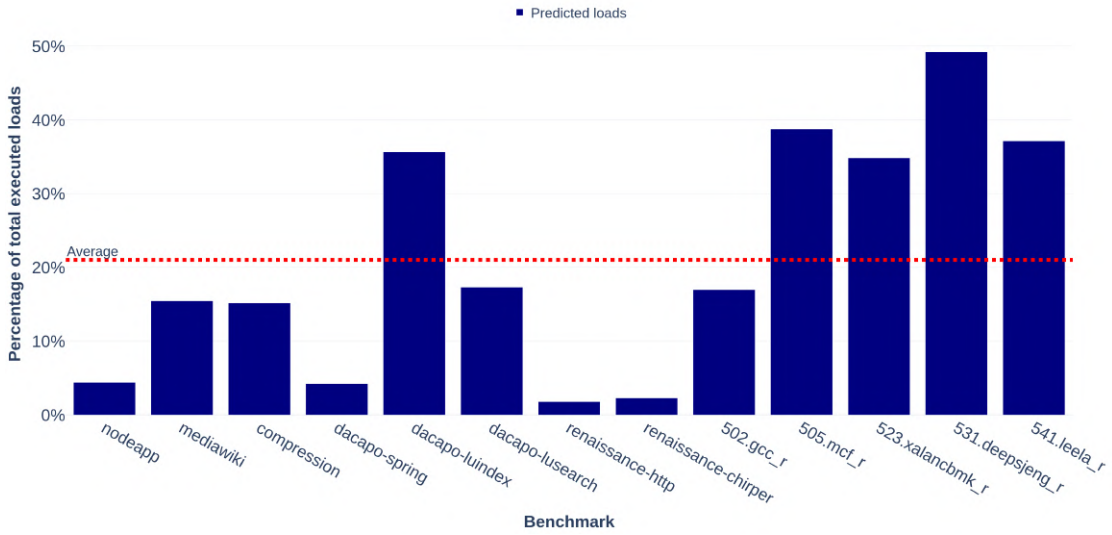


Figure 3.3: Percentage of executed loads that were predicted at least once for 13 different workloads.

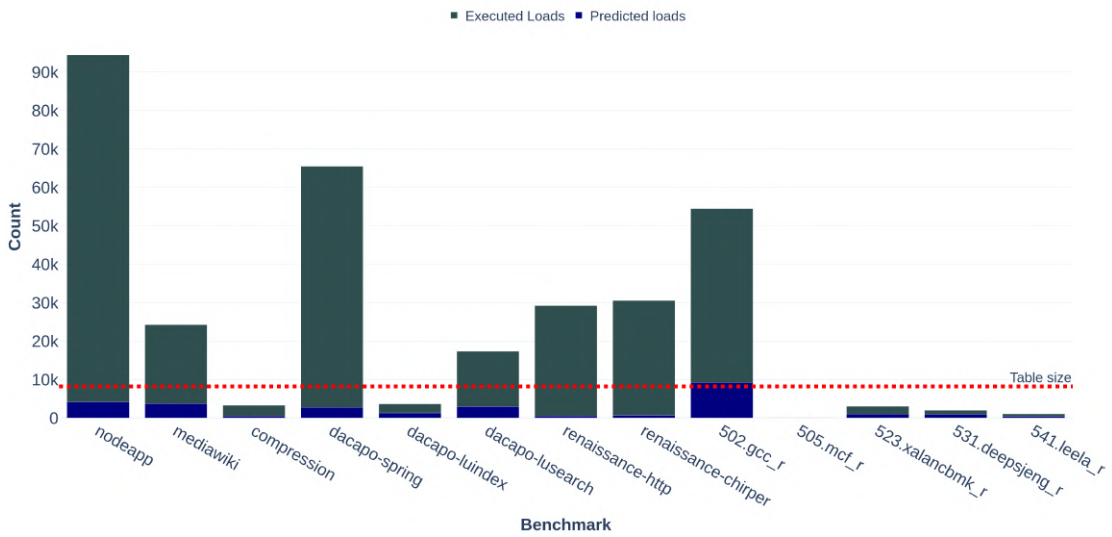


Figure 3.4: Count of executed loads and predicted loads for 13 different workloads.

predicted loads. When the number of candidate loads exceeds the table's capacity, newly encountered loads can push out entries that have already been trained or remain useful. Consequently, attempting to predict every load may reduce coverage and discard potential speedup rather than improve performance. For example, the "dacapo-spring" benchmark has around 65K loads, of which only 2.7K are predictable. If we now try to predict all loads, they often push each other out of the table, reducing the number of predictable loads we can keep.

To better understand how the predicted instructions behave and how the training phase affects the predictor going forward, we examine a series of snapshots of the prediction table.

3.2.3 Table Behavior

Previously, we looked at the predictor's behavior across entire simulations; now we want to inspect the table's behavior over time within a single simulation. Additionally, we want to examine how training and predictable loads affect the table's capacity. To that end, we recorded a snapshot of our 8K-entry table every 1,000 activations of the value predictor. In each snapshot, we counted how many loads are actively predictable, in training, or completely untrained. For a load to count as active, it must have a confidence of 50 so that it is predicted when the instruction is executed. Entries with a confidence of 0 are untrained. All other entries with a confidence between 0 and 50 are currently in training.

For brevity, the following analysis focuses on the nodeapp benchmark as a representative example; the same behavior was observed across the other benchmarks. Figure 3.5 shows 21K snapshots of the 8K table. In blue, we have the untrained loads; in orange, those currently in training; and in green, those that are active. The table is already full in the first snapshot because a warm-up phase precedes the simulation and populates it. We observe that most of the table capacity is consumed by untrained and training loads. This leaves only a small percentage available for loads that are actively being predicted.

One pattern we can observe is what happens when many new instructions, whose values were previously not in the table, are executed. Figure 3.6 shows this occurring around snapshot 5500. In Figure 3.6b, we simulated the same benchmark with a 130K table size, guaranteeing that no loads are pushed out of the table. Shortly after the 5500th snapshot, the overall count of executed loads increases as new entries are added to the table. While the new loads have little impact on the big table, Figure 3.6a shows that they have a much bigger impact on the smaller table. Initially, we have almost 2000 more untrained loads, as all the new executions are added to the table. This reduces the number of trained and active loads. In particular, the active loads go from almost

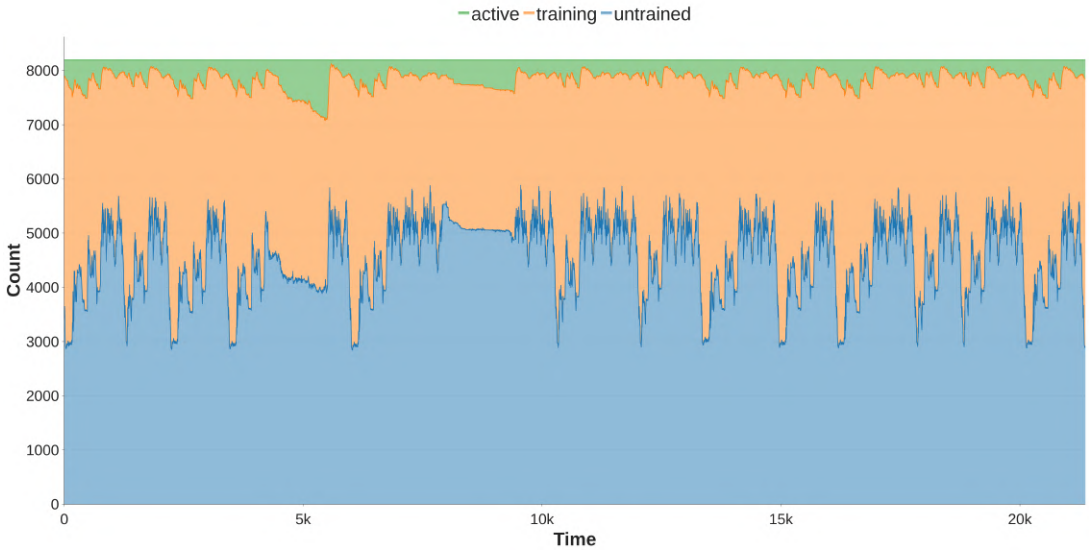


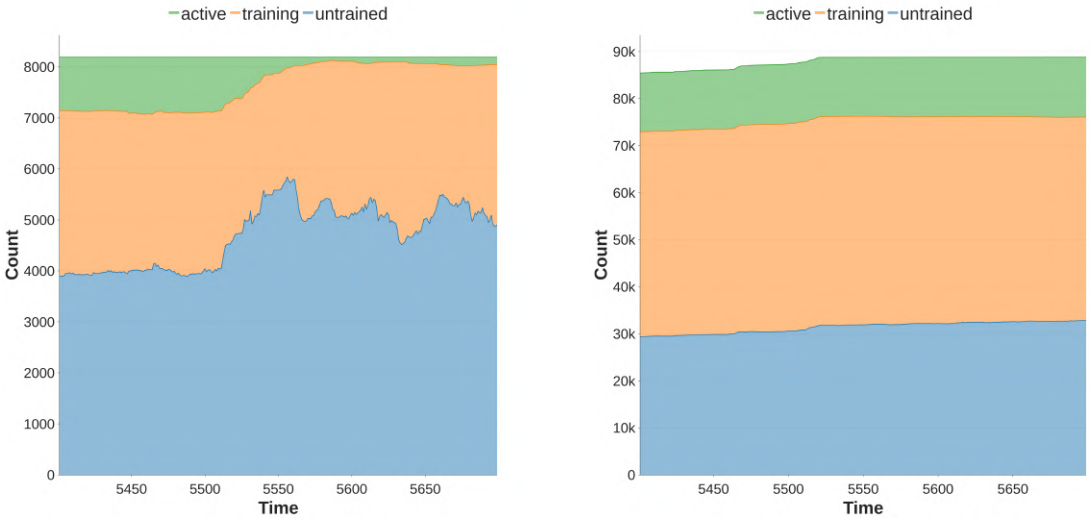
Figure 3.5: Table-Snapshots over time for the benchmark nodeapp

1000 beforehand to around 100. Because all these loads are fully pushed out of the table, they must go through the full training process again before becoming active. As a result, the table takes a long time to recover. In this case, Figure 3.5 shows that the table never reaches a point with 1000 active entries. This demonstrates that many new load executions within a short time frame reduce the efficiency of a standard-sized predictor by pushing active entries out, thereby requiring more training.

Overall, we observe that only a small fraction of the entries are used for active prediction, while the rest are used for training.

3.3 Conclusion and Goals

The analysis in Subsection 3.2.2 shows that only a small fraction of all dynamic loads can be predicted. Across the 13 benchmarks, only 21% of loads are predicted at least once on average. The remaining 79% consume table capacity during training but never reach the confidence threshold required for prediction. This means that a large part of the table capacity is wasted on instructions that never contribute to any speedup. Furthermore, even for predictable loads, most table entries are used for training rather than active prediction. As the table snapshots in Subsection 3.2.3 show, the majority of entries at any given time are either untrained or still in training. Only a small percentage reaches the confidence threshold for active prediction. This effect is even stronger because accurate prediction requires a high confidence threshold. Our experiments show that



(a) Size 8K

(b) Size 130K

Figure 3.6: Cutout of table snapshots for the nodeapp benchmark, comparing an 8K-entry table (a) with a 130K-entry table (b). The 8K table shows a sharp drop in active entries when many new loads arrive, while the larger table remains stable.

thresholds below 50 lead to frequent mispredictions, degrading performance across several benchmarks. Additionally, bursts of new load instructions can temporarily dominate the table, displacing previously trained entries. Subsection 3.2.3 shows that a sudden increase in new loads causes a sharp drop in active entries, which is especially true for smaller table sizes. These evicted entries must then be retrained from scratch. This reduces the effective coverage and increases the time until the predictor returns to a stable state. These observations show that the main problem is not just the absolute size of the prediction table, but how the capacity is used. A large part of the storage is occupied by loads that are either unpredictable or still training, while very few entries generate speedup. Because of this, we want to design a system that separates the early training phase from the main prediction table, so that only loads with stable behavior can access the main table. This suggests a different approach: rather than increasing table size, we should separate the initial training phase from long-term prediction storage. Only loads that demonstrate stable, predictable behavior should be allowed to occupy space in the main prediction table. Based on this insight, the predictor design should satisfy the following goals:

1. Better table utilization. The predictor must maximize the utilization of the prediction table for active, high-confidence values by preventing untrained or unpredictable loads from flooding the table and by reserving capacity for instructions that will likely contribute to the overall speedup.

2. Control the entry rate. The design must regulate the rate at which new load instructions enter the table, because too many entries at once can overflow the table and overwrite valuable trained data. The predictor should filter out long sequences of new loads that are unlikely to be predictable.

3. Low hardware overhead. The additional mechanism must introduce low access latency and minimal hardware area overhead. The modification must be lightweight enough to sit in the front end of the pipeline, without creating a new bottleneck. Additionally, it cannot be too large, as it would otherwise consume the resources gained by more efficient use of the prediction table.

4. Maintain or improve performance. Even with fewer resources, the predictor should deliver at least as much speedup as a conventional stride predictor with a larger table.

To reach these goals, we propose a two-level organization that integrates with the stride load value predictor. A small pre-filter table sits ahead of the main prediction table and handles the initial training phase. Only loads that reach a predefined

confidence threshold within a short time window are forwarded to the main table. This way, the filter decouples the initial training phase from long-term storage and protects the main table from unpredictable load spikes. The detailed design and implementation are presented in Chapter 4.

4 Design and Implementation

In this chapter, we present the design of the two-level predictor that fulfills the goals outlined in Section 3.3. We start by implementing the stride predictor in gem5, then continue with the design of the filter, and finally describe how we integrate our filter design with the stride predictor to create a two-level predictor implementation.

4.1 Stride Predictor Base

The basis for our value predictor is the "value-pred" branch of the "gem5-fdp" repository by Dr. David Schall [Scha]. It implements a stride load value predictor for the gem5 O3CPU model. It consists of a prediction table, the in-flight queue, and two key functions. The prediction table implementation uses an 8-way associative cache with a Least Recently Used (LRU) replacement strategy. Each entry in the prediction table includes the standard fields required for stride prediction: the value, stride, confidence counter, and replacement-algorithm information.

The in-flight queue records instruction instances that are currently in the pipeline between value prediction in the rename stage and table updates during the commit phase. It is necessary to correctly calculate how many strides to add to the value in the table, since this value is updated only when instructions reach the commit stage.

The first key function is the "lookup" function, which is called during the rename stage of the pipeline. Here, the processor filters for instructions it predicts, all of which are loads with a single integer return register. Afterward, it uses the instruction PC to generate a tag to access the table and check for a valid entry. If no entry is found, the processor marks this instruction instance as not predicted. If an entry is found, its confidence is compared against the threshold to determine whether it is ready for prediction. If so, the predictor calculates the prediction value by using the following formula.

$$\text{prediction value} = \text{value in table } + (\text{number of in-flight instances } + 1) \cdot \text{stride}$$

If the entry is a constant value, the predicted value equals the table value, because the stride is 0. Otherwise, we add the stride multiplied by the current number of in-flight instances of this instruction plus the current one to calculate the prediction value.

Afterward, this instance of the instruction is marked as predicted, and a reference to the result is added to the instruction. Finally, regardless of whether an entry exists, the processor adds this instruction instance to the in-flight queue to account for the current instruction, then continues executing the remaining rename stage.

The second key function is the "update" function, which is called in the commit stage of the pipeline. Here, the processor filters for the same instruction type and then accesses the table to check for a valid entry. If no valid entry matches the instruction's computed tag, the replacement policy selects a victim entry and allocates a new entry initialized with the correct value and a confidence counter of 0. As this is the first instance of this instruction, the stride value is 0. If a valid entry exists, the predictor compares the entry's value, adjusted for the stride, with the actual value and checks whether they are identical. If they are identical, the predictor increments the confidence counter until it reaches the confidence threshold. Otherwise, the confidence threshold is reset, and the stride value is recalculated, using the actual result of the instruction. Additionally, the processor must squash the pipeline and restart with the execution of the mispredicted instruction.

Before proceeding with the rest of the commit stage, the predictor updates the in-flight queue by removing the entry at the front of the queue. This is always possible because all instructions go through the commit stage in order, so the entries are in the same order as the executing instructions.

Figure 4.1 provides an overview of the complete process. We access the prediction table in both the rename and commit stages of the pipeline using the two key functions. Each function updates the in-flight queue before continuing with the rest of the pipeline stage, ensuring the predictor correctly predicts stride patterns.

4.2 Filter Design

In this section, we first describe the general design idea behind the filter and then clarify what each entry in the filter requires to function. To sum up this section, we explain how this design fulfills the previously outlined goals.

4.2.1 Filter Concept

Our design introduces a dedicated filter that is positioned ahead of the main prediction table. The primary purpose of this filter is to determine how likely it is that an instruction will exhibit predictable behavior in the future. The design we chose for our filter is that of a downsized prediction table. The filter sits in front of the main prediction table and handles the initial training phase. Only instructions that reach a predefined entry threshold in the filter are forwarded to the main table. In the

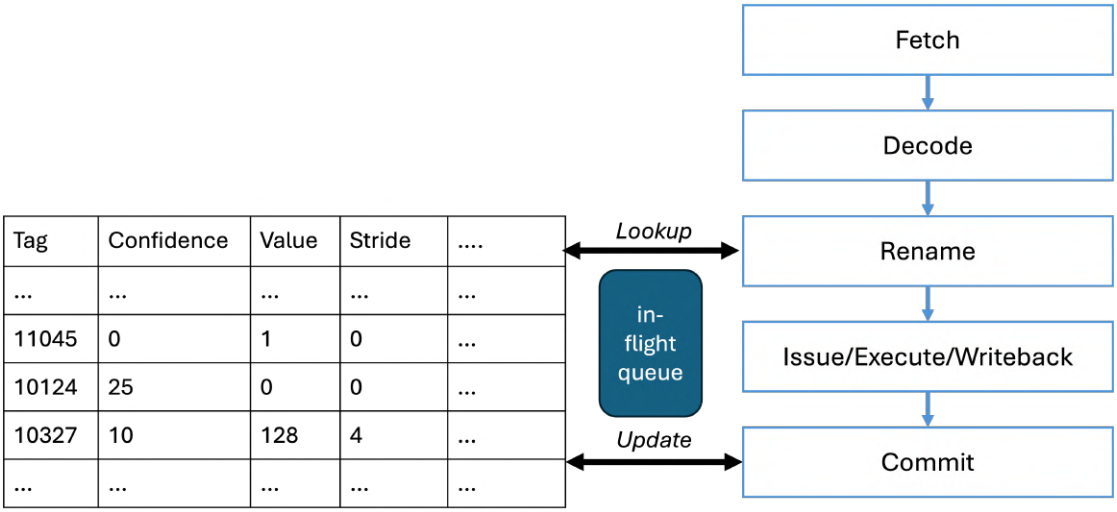


Figure 4.1: Graphic of the stride load value predictor implemented in gem5

remainder of the thesis, we refer to the threshold required to pass through the filter as the entry threshold, and the threshold for starting prediction in the main table as the confidence threshold.

When a filter entry's confidence reaches the configured entry threshold, the instruction is inserted into the main prediction table. The main table entry is initialized with the filter's current value, stride, and confidence, which equals the entry threshold. From that point on, the main predictor tracks the instruction, and the entry is marked as invalid in the filter. This ensures that new instructions occupy unused entries first, rather than pushing out loads that have already started training. As a result, the main prediction table is primarily occupied by instructions that have already shown stable behavior, while the filter absorbs the initial training phase.

By varying the filter's entry threshold, we can control its selectivity. If we employ a very low threshold, many instructions reach the main table. This increases overall coverage, as the main table has more loads to train, but it also increases the risk of it getting overloaded with only weakly predictable loads. On the other hand, a higher entry threshold significantly limits overall coverage while reserving more space in the main table for actively predicting loads. In 5.2.2, we determine exactly how the entry threshold affects the predictor's behavior and which configurations provide the best results.

4.2.2 Entry of Filter

Ideally, the filter entries would be as small as possible to minimize space overhead, which is one of our design goals. To identify each instruction, each entry must include a tag, which we keep the same length as in the main table for consistency. Additionally, because the filter is implemented as a table, we need the information required for the replacement strategy. Because the filtering process depends on how quickly an entry starts training, each entry needs all the information required for training. This depends on the prediction scheme used by the main table, as both must use the same algorithm. In this case, the base implementation is a load-stride predictor, meaning the entry requires both a value, a stride, and a confidence counter. Both the value and stride fields need to be full-size, but the confidence counter can be smaller since we only need to record the start of training and therefore does not need to count as high as during a full training run. Overall, we can conclude that an entry in the filter table has almost the same format as in the main table, except with a smaller confidence counter, which can save a few bits. For example, if we use a confidence threshold of 50 in the main table and an entry threshold of 5, we need 3 fewer bits for the filter entry.

4.2.3 Table as Filter

In this section, we review the goals described in Section 3.3 and explain how our filter fulfills them.

1. By using a small prediction table as a filter for the main table, we can identify instructions with a high frequency of predictable executions. These show a rapid increase in confidence and can therefore be forwarded to the main table. If an instruction's confidence does not increase fast enough within a short time, it is pushed out of the table when a new instruction arrives and is evicted by the replacement strategy.

2. It also solves the problem of long sequences of new, unpredictable instructions that overwhelm the prediction table. Because they are unpredictable, they cannot reach the main table, where the instructions that have already been trained for a long time and are actively predicting are stored. The new instructions can at most push out training instructions that are still in the filter table and therefore at the beginning of the training phase.

3. As a filter, the table is intentionally small, thereby producing minimal hardware area overhead. It is also only involved in the training process and does not introduce additional latency during prediction, where speed matters most.

4. By meeting these points, the filter should reduce the predictor's resource requirements while still achieving comparable performance. Chapter 5 investigates how our filter design performs in detail.

4.3 Integration into Stride Predictor Implementation

In this section, we implement our filter design together with the stride predictor, combine them into a single two-level value predictor, and explain the changes made to the "lookup" and "update" functions.

The "update" function still starts by accessing the main prediction table, and if it finds a valid entry for the instructions tag, it proceeds exactly as previously explained for the stride predictor. The changes begin when no entry is found in the main prediction table. The predictor then accesses the filter table and checks for a valid entry using the instructions tag. If an entry already exists, the filter updates the confidence and stride as in the main predictor: on a correct prediction, the confidence is incremented; on a misprediction, the confidence is reset, and the stride is recomputed. When a filter entry's confidence reaches the configured entry threshold, the instruction is inserted into the main prediction table. The main table entry is initialized with the filter's current value, stride, and confidence, which equals the entry threshold. From this point on, the main predictor tracks the instruction. The corresponding filter entry is then set invalid. If no entry that matches the tag exists in the filter, the predictor allocates a new filter entry with confidence 0 and stores the initial value and stride. When the filter table is full, the predictor evicts a victim entry and replaces it with the new instruction. Finally, in both cases, the corresponding instruction instance must be removed from the beginning of the in-flight queue.

Because the filter is not used to generate predictions, it does not change the lookup operation. The lookup function continues to access the main prediction table directly, where only fully trained entries are eligible for prediction. The filter is used exclusively during the initial training phase and therefore never contains entries that are ready to generate predictions. Separating the filter from the prediction path leaves the lookup function's critical path unchanged. Consequently, the two-level predictor introduces no additional lookup overhead in the rename stage

The in-flight queue does not need to be duplicated for the filter table, because an instruction is never in both tables; the filter and main table can therefore use the same queue independently.

From the pipeline's perspective, the two-level organization does not change the existing lookup and update interfaces. The additional filtering logic is integrated into the value predictor's existing prediction and update paths, and no changes are required

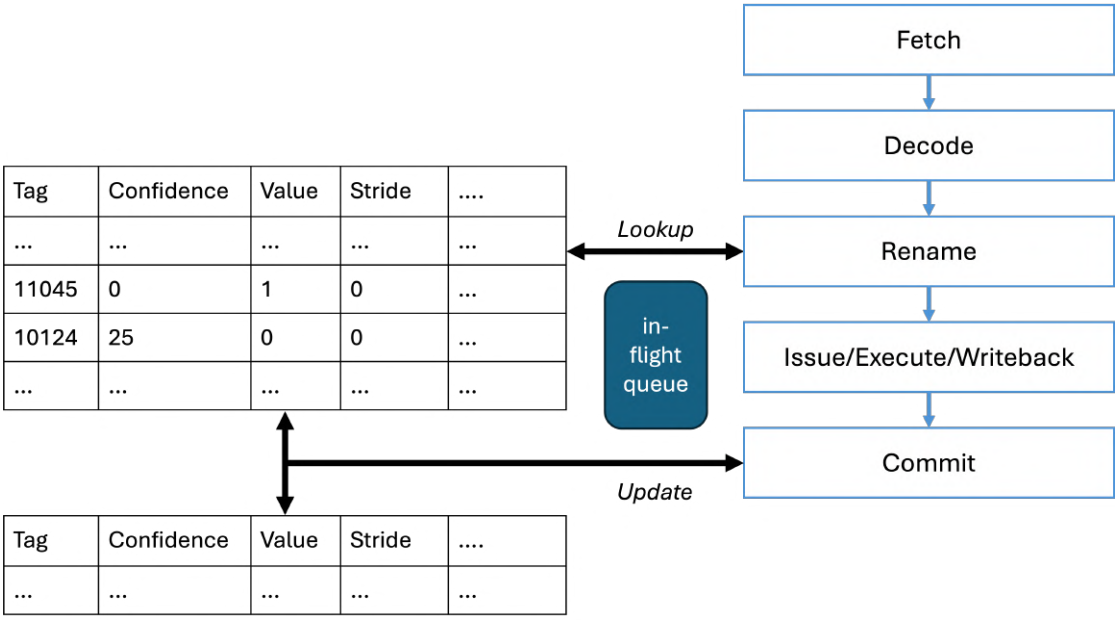


Figure 4.2: Structure and data flow of the two-level load-value predictor implemented in gem5. The filter table performs the initial training phase and selectively inserts predictable loads into the main stride prediction table, which generates and verifies the value predictions.

to the remaining rename and commit stages. The whole process can be seen Figure 4.2. The full implementation of the two-level predictor in gem5 is available on GitHub in the gem5-lvp repository [Lis].

5 Evaluation

In this chapter, we evaluate the proposed two-level value predictor in terms of its performance impact, storage efficiency, and table behavior. The primary goal of the evaluation is to determine whether the filter-based design can reduce resource usage while preserving the speedup achieved by the standard stride predictor. To this end, we compare the two-level predictor with the stride value predictor from Chapter 3 and analyze its behavior on the same benchmark set used in the motivation study. We first describe the concrete gem5 configurations used in all experiments. Afterward, we compare different settings of the two-level predictor to identify the optimal configuration that yields the best results. We then evaluate this configuration against the stride-value predictor to assess whether it improves the prediction table's efficiency and yields a comparable speedup even with fewer resources. Finally, we provide a brief overview of how the two-level predictor performs with entry thresholds other than the confidence.

5.1 Methodology

For all experiments in this chapter, we run a server benchmark suite comprising 13 benchmarks, as listed in Table 5.1. Dr. David Schall [Schb] provided all these workloads, along with a framework. To reduce simulation time, the experiments use representative execution regions via simpoints. For each benchmark, the simulator first fast-forwards to the selected region, then executes a 50 million-instruction warm-up phase before collecting statistics over the following 100 million instructions. The same procedure applies to all predictor configurations. Unless otherwise stated, the reported results are based on the selected region for each benchmark and are not derived from a full-program simulation [1]. [Schb]

All simulations use the CPU configuration modeled after the Intel Xeon Granit Rapids architecture, as shown in Table 5.2 [2]

[1] Script used for simulations: `https://github.com/dhschall/gem5-svr-bench/blob/main/scripts/run_important_simpoints.sh`

[2] Concrete configuration: `https://github.com/dhschall/gem5-svr-bench/blob/main/gem5-configs/all-simpoint-run.py`

Benchmark	Description
nodeapp	Node.js website [Schb]
mediawiki	PHP website [Schb]
compression	Google workload, data focused [Abe+24]
dacapo-spring, dacapo-luindex, dacapo-lusearch	dacapo, java benchmark suite [Bla+06]
renaissance-http, renaissance-chirper	renaissance, java benchmark suite [Pro+19]
502.gcc_r, 505.mcf_r, 523.xalancbmk_r, 531.deepsjeng_r, 541.leela_r	SPEC workloads [Sta17]

Table 5.1: Overview of the benchmarks used in the evaluation

System	Full-system ARM
Core	3GHz, 8-wide OoO, 567-entry ROB, 189-entry LQ, 120-entry SQ
Cache Hierarchy	L1I: 64KiB 16-way, L1D: 48KiB 12-way, L2: 3MiB 12-way, L3 8MiB 16-way
BTB	16K entry, 8-way associative
Branch Predictor	64KB TAGE-SC-L
Prefetcher	Data: Stride, Instructions: Fetchdirected + Tagged

Table 5.2: Overview of the CPU configurations

Unless otherwise specified, in all simulations that use an active value predictor, the stride predictor uses an 8-way associative 8192-entry prediction table with a 16-bit tag and uses LRU as the replacement strategy. The confidence threshold is set to 50, and we reset to zero on a misprediction. Each entry in the prediction table is 19 bytes, consisting of a 2-byte tag, an 8-byte value, an 8-byte stride, a 6-bit confidence counter, and 1 useful bit and 1 valid bit. This configuration is also used for the main table of the two-level predictor. The filter table is also an 8-way associative table, with a 16-bit tag and LRU replacement. However, when the entry threshold is low enough, one entry may require fewer bytes. All other filter parameters are determined during the optimization in Section 5.2.

For all speedup calculations, we use a no-predictor simulation as a baseline, with the same configuration as the stride and 2-table value predictor, except that the value predictor is turned off.

The data used in the following sections was either gathered from the integrated gem5 "stats.txt" file or from custom functions that store additional data in CSV files.

5.2 Comparison of Different Filter Configurations

The first series of experiments compares different configurations for the 2-table predictor, using the same main table to find the best combination.

5.2.1 Size of Filter

We start by varying the number of filter entries. To do so, we simulate the 2-table predictor across all possible sizes from 32 to 4096. This yields 8 distinct data points, since we can only create table sizes that are powers of two. We chose a maximum of 4096 entries, the size one below the main table, because the bigger the filter gets, the less space we save by using it. In this case, with a main table of 8192 entries, an additional filter of the same size would not be more efficient than a simple stride predictor of size 8192, since the additional filter would consume roughly the same amount of storage without providing additional filtering benefit.

We simulated this experiment across a range of filter entry thresholds. In Figure 5.1, we show the predictor speedup at entry threshold 4 for all benchmarks, with the average shown as the red dotted line. For brevity, we will not examine the other entry thresholds, which yielded the same results regarding the dependency on size. The highest average speedup, 5.7%, is achieved with 256 table entries in the filter table. This is primarily due to the "dacapo-luindex" benchmark, which achieves significantly greater speedup with the 256-entry table than with any other size. For larger filters, although individual benchmarks vary, the average speedup remains slightly below the peak, at a nearly constant 5.6%. Below 256 entries, the predictor shows significantly less speedup, dropping to 2.8% for a filter size of 32.

To minimize the additional resources required by the two-level design, we use a 256-entry filter table in the following experiments.

5.2.2 Entry Threshold to Main Filter

The next variable we examine is the entry threshold for the main predictor. To do so, we simulated the 2-table predictor with a 256-entry filter across a range of thresholds from 1 to 32. We chose not to evaluate a higher entry threshold because the filter's function is not to train the entries but only to determine whether they are predictable. For this, it suffices to evaluate only a small portion of the training phase and to continue most of the training in the main table. Because the entry threshold determines how many entries reach the main table, and therefore also how many values we need to save, we evaluated the entry thresholds for two different-sized main predictor tables, namely 256 and 8192 entries.

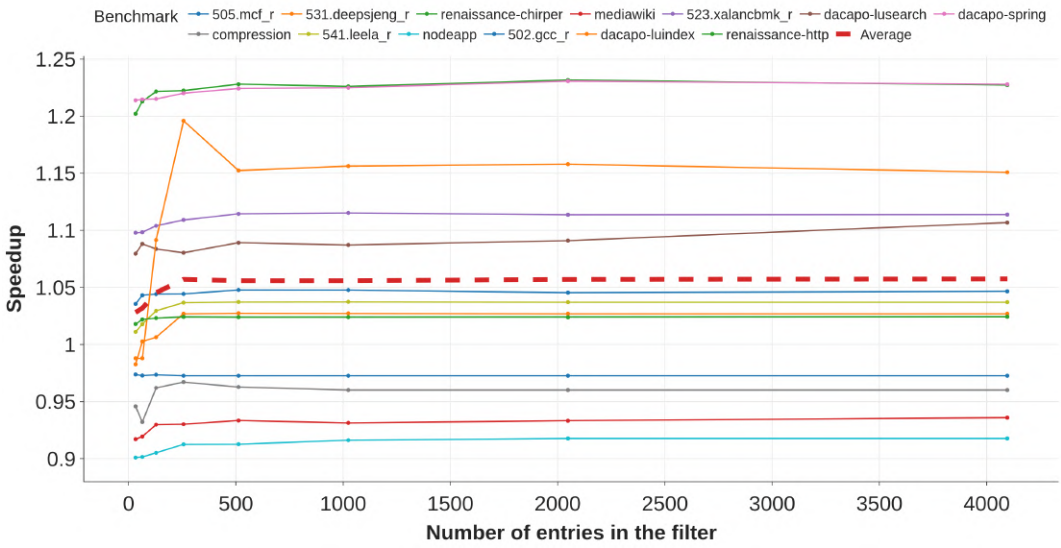


Figure 5.1: 2-table predictor speedup of 13 different benchmarks compared to the no predictor baseline, plotted over the number of prediction table entries in the filter table using an entry threshold of 4.

In Figure 5.2, we display the averages for both experiments. In orange, we can see that the 256-sized predictor achieves its best average speedup of 6% at an entry threshold of 4. The blue graph, which represents the average for the 8192-entry table, has the highest speedup of 5.3% at an entry threshold of 1. This is because with a smaller main prediction table, a too-low entry threshold does not filter out enough instructions to prevent the table from overrunning. On the other hand, for a larger main table, a too-high entry threshold might be overly strict, preventing the table from reaching its full capacity and thereby reducing coverage.

Because both have advantages, we will examine how the entry thresholds of 1 and 4 perform across all subsequent experiments.

5.3 Comparison to Stride Predictor and Different Sizes of Main Table

So far, we have mainly looked at different filter configurations, kept the main table largely the same, and compared them against a no-predictor simulation. Next, we directly compare the two-level predictor with the baseline stride predictor for different main table sizes.

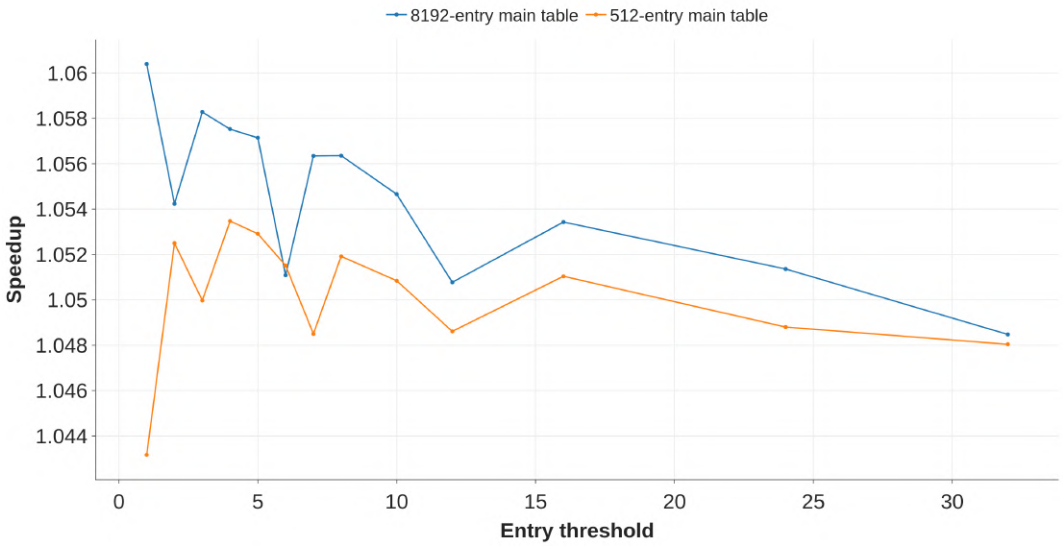


Figure 5.2: 2-table predictor speedup with 8192-entry and 512-entry main tables, plotted over the entry threshold to the main table for a filter size of 256 entries.

5.3.1 Prediction Table Efficiency

To better understand how the 2-table predictor affects usage and, therefore, the efficiency of the main table, we study a timeline of prediction-table snapshots. Subsection 3.2.3 explains the exact mechanism for recording the snapshots. In summary, we take snapshots of the prediction table that count the number of active, training, and untrained entries it currently contains. In the following sections, we examine the two-level predictor in detail using entry thresholds of 1 and 4 for main table sizes of 256 and 8192 entries. We compare these to the stride predictor for sizes 512 and 8192. We chose 512 for the smaller table for stride prediction and 256 for the main table of the two-level predictor with a 256-entry filter, because both have similar space requirements, making comparison easier.

256 Entries

Figure 5.3 shows the snapshots of the stride predictor with a 512-entry table. Almost all the table's capacity is used for untrained loads because, with such a small table, almost no load can train for long before new instructions push it off the table. The two-level predictor avoids this problem by allowing only a portion of the loads into the main table, thereby reducing excessive trashing. This is visible in Figure 5.4, which depicts snapshots of a two-level predictor with a 256-entry main and filter table and

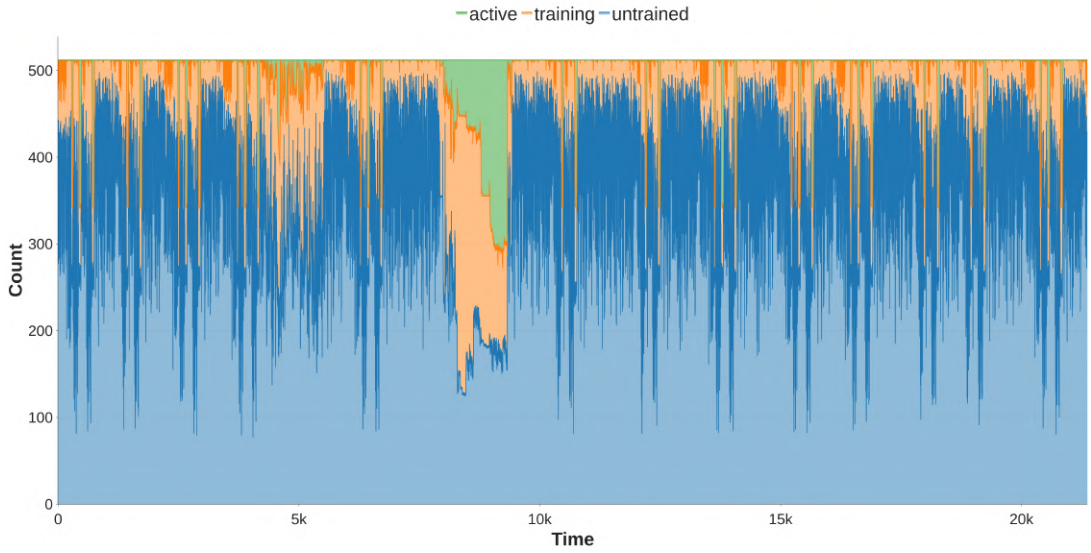


Figure 5.3: Table snapshots over time for a 512-entry stride predictor using the nodeapp benchmark. The table shows a larger fraction of entries (almost all) in the untrained state compared to the two-level predictors in Figure 5.4 and Figure 5.5.

an entry threshold of 1. For this configuration, almost all the loads are in the training phase. While this is better than the stride predictor, almost no loads still reach an active state. To achieve this, the predictor needs to be even more selective by increasing the entry threshold. Figure 5.5 shows that with an entry threshold of 4 for the two-level predictor, the percentage of loads in the active state increases significantly compared to the lower entry threshold and the stride predictor. This comparison between Figure 5.3 and Figure 5.5 shows that with the right filter configuration, we can drastically increase the percentage of loads that finish training, without increasing our space requirements, as the number of entries in the filter and main table of the two-level predictor combined equals that of the prediction table of the stride predictor. One entry in the main prediction table uses 19 bytes, of which 6 bits represent the confidence counter for a confidence threshold of 50. In the filter, we use an entry threshold of 4, which requires 3 bits. Therefore, we save 2 bits per entry, for a total of 64 bytes. As we can likely only address memory byte-wise, we cannot use these 2 bits and therefore keep the same 19 bytes per entry, meaning our space requirements stay the same.

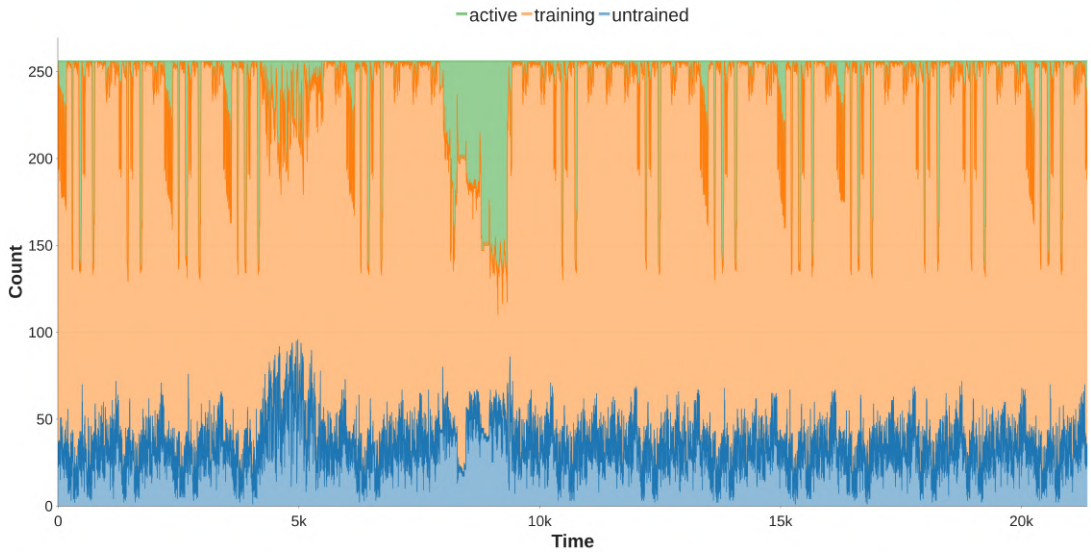


Figure 5.4: Table snapshots over time for the 256-entry two-level predictor with an entry threshold of 1 using the nodeapp benchmark. The table shows a larger fraction of entries in the training state (almost all) compared to the stride predictor in Figure 5.3, but significantly fewer in the active state than the 2-level predictor in Figure 5.5.

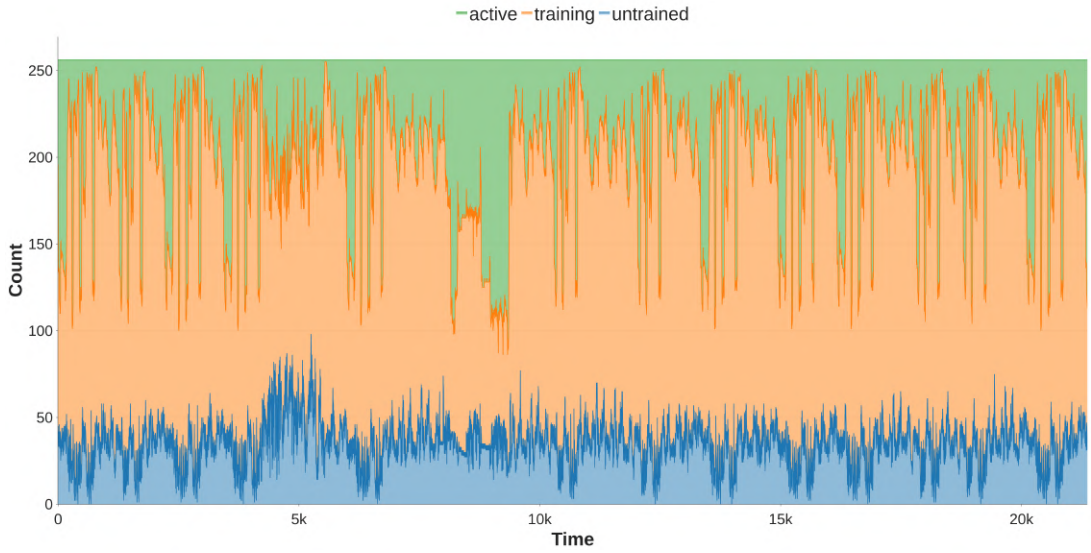


Figure 5.5: Table snapshots over time for the 256-entry two-level predictor with an entry threshold of 4 using the nodeapp benchmark. Compared with the stride predictor in Figure 5.3 and the two-level predictor with an entry threshold of 1 in Figure 5.4, the filter significantly increases the fraction of active loads.

8192 Entries

Compared to the snapshots in Figure 3.5 of the stride predictor prediction table in Chapter 3, the main table of the two-level predictor with an entry threshold of 1 in Figure 5.6 shows a much higher percentage of the table being used for active prediction and fewer entries that are completely untrained. Comparing this to the two-level predictor with an entry threshold of 4 in Figure 5.7, we see that it never reaches full capacity, especially at the start of the simulation, with many entries remaining free. This is because the higher entry threshold makes the filter too strict and does not let enough loads into the main prediction table. What stands out is that in 5.7, especially towards the end, a lot more entries are active than in Figure 5.6. However, many of these are old entries that were never pushed out because the table never reached its full capacity and therefore no longer contribute to the speedup. Comparing the actual coverages, we can see that with 33.4% coverage over 33.2%, the two-level predictor with 1 as the entry threshold barely predicted more values.

These table snapshots, for both the 256-entry and the 8192-entry two-level predictor, justify the results in Subsection 5.2.2. Entry thresholds around 4 are better for small main prediction tables, while 1 is better for larger ones.

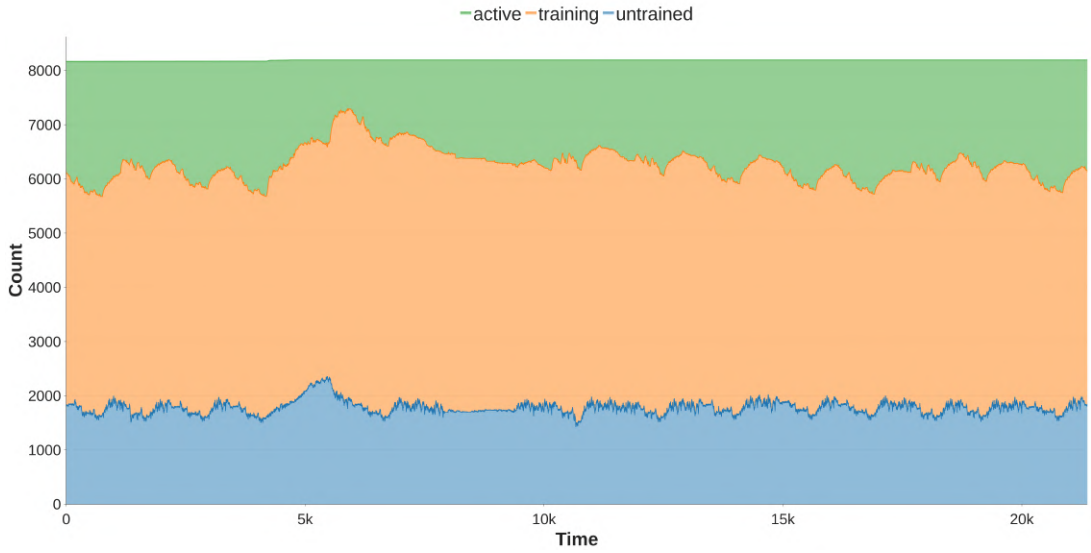


Figure 5.6: Table snapshots over time for the 8192-entry two-level predictor with an entry threshold of 1 using the nodeapp benchmark. Compared with Figure 3.5, a higher fraction of entries is actively used for prediction, and fewer entries are completely untrained.

5.3.2 Handling Bursts of New Loads

In this section, we examine how the filter responds to a rapid increase in newly encountered load instructions. To this end, we analyze an event in the nodeapp benchmark around snapshot 5500 for a predictor table size of 8192. We discussed the baseline stride predictor's behavior in this case in Subsection 3.2.3 and show it in Figure 3.6a.

At this point in the execution, many load instructions that have not previously been stored in the prediction table are executed in a short period. In the baseline stride predictor, these instructions are inserted directly into the main prediction table. They therefore occupy entries even though they are still untrained. If the table is close to its capacity, the new entries replace loads that have already been trained or are currently being predicted. As shown in Figure 3.6a, this leads to a strong decrease in the number of active entries. The evicted loads must be retrained before they can be used for prediction.

Figure 5.8a shows the same part of the execution for the two-level predictor with an entry threshold of 4, and Figure 5.8b shows it for an entry threshold of 1. In both cases, newly encountered loads are first inserted into the filter table. They are transferred to

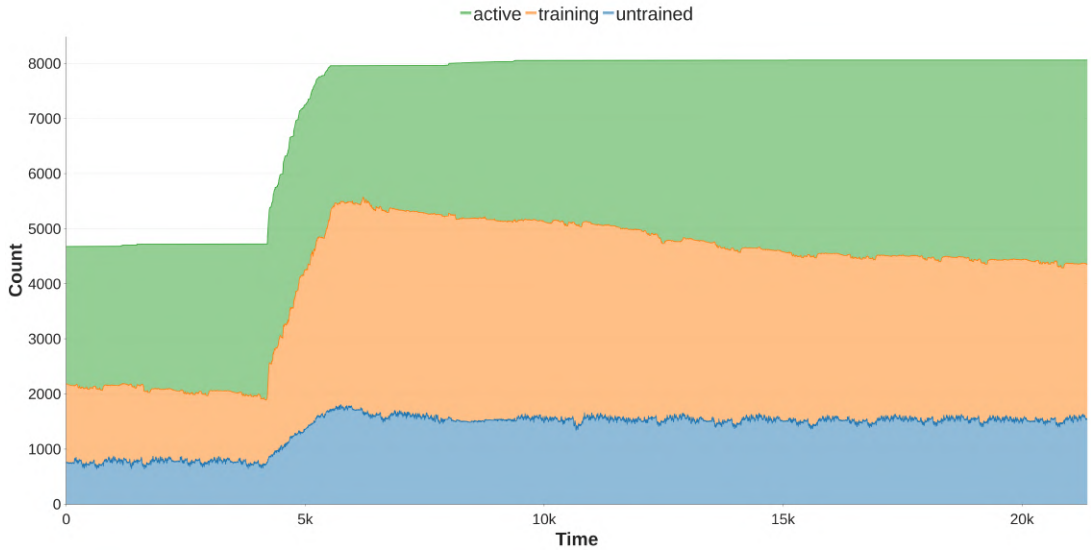


Figure 5.7: Table snapshots over time for the 8192-entry two-level predictor with an entry threshold of 4 using the nodeapp benchmark. Compared with Figure 3.5, a higher fraction of entries is actively used for prediction, while the strict filtering leaves the table partially empty.

the main prediction table only after reaching the configured entry threshold. Loads that do not reach this threshold remain in the filter or are replaced by other entries. Consequently, an increase in new load instructions does not directly affect the main prediction table.

Compared with the baseline predictor, the two-level predictors retain nearly all previously trained entries throughout this interval. The number of active entries remains largely unchanged, while the filter handles newly encountered loads. This prevents a short sequence of new instructions from replacing entries that already contribute to the predictor's performance.

This example clearly shows how the filter reduces the impact of sudden increases in the number of candidate loads. This behavior is particularly relevant for smaller prediction tables, where new entries are more likely to replace useful ones. For larger tables, the difference between the two predictors is expected to be smaller because the main table can accommodate more loads without causing extensive replacements.



(a) Entry threshold 4

(b) Entry threshold 1

Figure 5.8: Zoom-in on a rapid increase in newly encountered loads for the two-level predictor using the nodeapp benchmark, comparing an entry threshold of 4 (a) with an entry threshold of 1 (b). In both cases, the filter absorbs the rapid increase.

5.3.3 Speedup

We simulated both the stride predictor and the two-level predictor with entry thresholds 1 and 4 for main table sizes ranging from 32 to 130K entries, using powers of two as intermediate points. In Figure 5.9, we plot the average speedup across all 13 benchmarks for the three simulated predictors. The x-axis counts the number of entries in the prediction table for the stride predictor or the combined number of entries in both the filter and main table for the two-level predictor. In blue, we see the stride predictor, in orange, the two-level predictor with an entry threshold of 1, and in green, the two-level predictor with an entry threshold of 4.

We can observe that for main table sizes below 2048 entries, the two-level predictor with an entry threshold of 4 is the fastest and achieves the highest speedup. It is followed by the other two-level predictor, and lastly, the stride predictor. For these small main tables, the strict two-level predictor works especially well, as it can restrict the flow into the main table to such an extent that many more entries reach the prediction stage. However, for more than 16K entries in the main table, the two-level predictor with an entry threshold of 4 also loses to the stride predictor, as it restricts the main table too much, resulting in too low coverage. Because of this, even for big table sizes, the speedup does not improve. For larger main tables with up to 32K entries, the less-strict two-level predictor performs best overall. After that, it is also too strict to use the tables to their full capacity, and for the largest table is barely slower than the stride predictor.

Overall, we conclude that using the two-level predictor significantly increases the speedup. At the most extreme point, we achieve a speedup comparable to that of a stride predictor while using only 9.6% of the space requirements. Both the stride predictor with 8192 entries and the two-level predictor with an entry threshold of 1 achieve a speedup of 1.05, while the two-level predictor requires only 512 entries in the main table and 256 in the filter. As established in Subsection 5.3.1, each entry in a filter with an entry threshold of 4 is also 19 bytes. Thus, the two-level predictor has a combined table size of 15KB, and the stride predictor has a table size of 156KB, saving us 154KB.

5.3.4 Coverage

Figure 5.10 compares the average coverage of the stride predictor with the two-level predictors with entry thresholds of 1 and 4. We plot the coverage for all powers of 2, from 32 to 130K entries in the main predictor over the combined number of entries in the filter and main prediction table, or only the prediction table for the stride predictor. In blue, we see the stride predictor, in orange, the two-level predictor with an entry

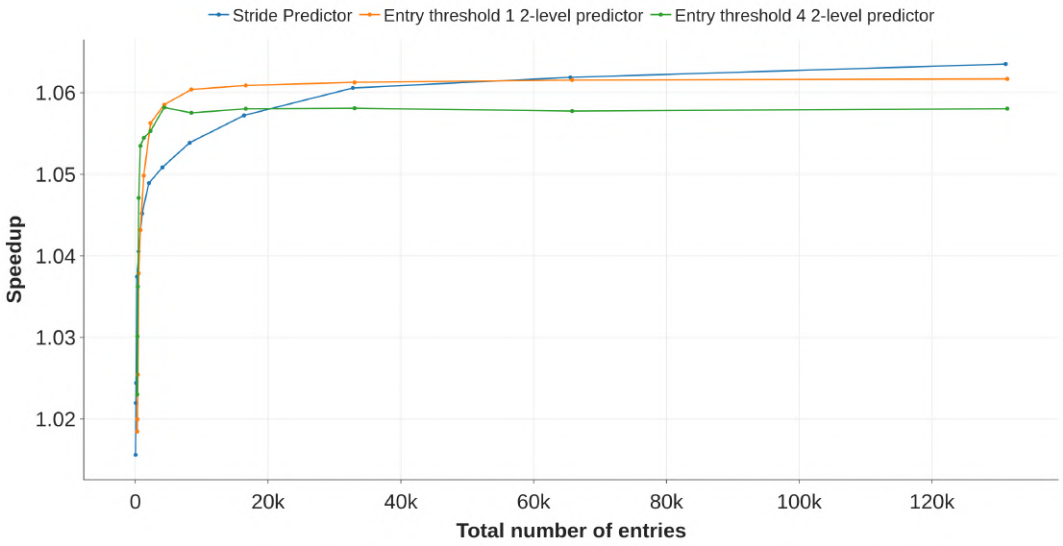


Figure 5.9: Average speedup of the stride predictor and the 2-level predictor with entry thresholds of 1 and 4, compared with the baseline (no predictor) as a function of the combined number of entries in the filter and the main prediction table.

threshold of 1, and in green, the two-level predictor with an entry threshold of 4.

For fewer than 4096 entries in the main table, the two-level predictor with an entry threshold of 4 achieves the highest coverage, followed by the other two-level predictor. This is because, with the high-selectivity filter, the small main table is less crowded, and loads have a better chance of reaching the prediction stage. For main-table sizes between 4096 and 16384 entries, the two-level predictor with the less strict filter has the highest coverage, while the stride predictor overtakes the strict two-level predictor at 8192 entries. For anything larger than 4096 entries, the strict two-level predictor is too selective and therefore doesn't use the full capacity of the main tables. This limits this predictor to a maximum coverage of 31% regardless of table size. The lenient two-level predictor can achieve better coverage up to 16384 values, but it also restricts the main table too much at larger sizes and cannot achieve coverage above 34%. For all larger prediction tables, the standard stride predictor achieves the best coverage, since the main table is large enough to accommodate most predictable loads even without filtering. This shows that the smaller the main table, the stricter the filter must be to avoid trashing and achieve the best coverage. The larger the prediction table is, the more instructions the predictor should let into the main table, since even with low efficiency, enough entries are available to predict as many instructions as possible.

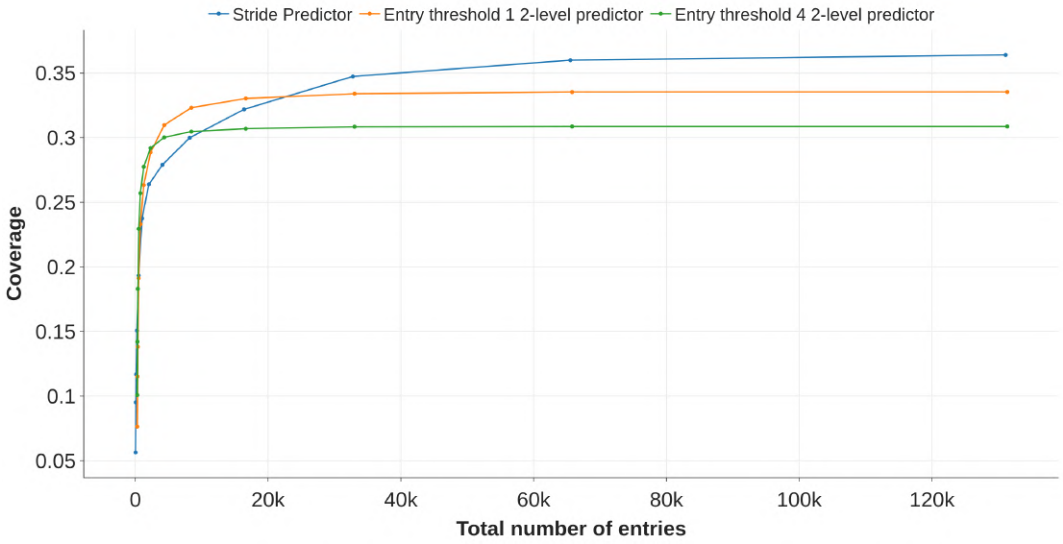


Figure 5.10: Average coverage of the stride predictor and the 2-level predictor with entry thresholds of 1 and 4, compared with the baseline (no predictor), over the combined number of entries in the filter and the main prediction table.

5.4 Other Entry Thresholds

In addition to evaluating different configurations of the two-level predictor, we briefly explored alternative criteria to replace confidence as the entry threshold for the second predictor. We wanted to focus less on how likely they are to be predictive and more on how much time these specific loads would save. To achieve this, we choose two criteria. The first is the number of L1 cache misses, since load savings mostly come from L1 cache misses [OAM18]. The second value is the time between the rename stage, where the value is predicted, and the commit stage, when the value is compared to determine whether it was correct or mispredicted. By using these as the entry threshold, we could focus our predictor on critical loads that are likely to yield the greatest speedup. However, the simple implementation we employed, which replaced one threshold with another, did not yield any performance improvements. This is likely because the simple criteria we chose are not yet precise enough to indicate future speedup potential. While an L1 cache miss indicates a long wait, it does not necessarily mean that this will be the case in the future, as the value is likely to be loaded into the cache. Also, the time between prediction and confirmation could be heavily influenced by other instructions that the inspected instruction must wait for. To make other thresholds

work, the threshold would need to be a more precise indicator of future criticality and might also need to account for predictability. In Chapter 8, we discuss more about additional methods that could be used in the direction of criticality-based prediction.

6 Related Work

The general idea of value prediction was first proposed by Lipasti et.al. [LWS96; LS96]. In this work, they first introduce the concepts of value locality and value prediction, and propose a preliminary design for a load-value predictor. Independent of Lipasti et.al., [GM98] proposed value prediction for general-purpose and floating-point registers and presented a tagged stride predictor. These works establish the general idea of value prediction without delving deeply into how to achieve efficiency improvements.

State-of-the-art value predictors achieve peak performance improvements by combining existing stride-based designs with modern branch prediction techniques [Sez18; Sak+]. VTAGE [PS14] is one of the first to use this strategy, by adapting the design used in the Indirect Target TAgged GEometric history length (ITTAGE) [SM06] branch predictor and using it in tandem with a stride predictor to achieve up to a 65% speedup. They combine multiple prediction schemes to maximize their coverage and generally try to predict as many values as possible. In contrast, we aim to limit the number of instructions our predictor attempts to predict.

Two propositions stand out for reducing the resources required for value prediction. The first direction is proposed in [OAM18], which introduces the Address-first Value-next Predictor with Value Prefetchin (AVPP), an adaptation of load value prediction that serves as an efficient alternative. AVPP first predicts load addresses, then uses them to predict their values. To do so, it uses a non-speculative value table that stores the load values and is accessed via the predicted address. By predicting addresses for future instances of the same load access, it prefetches the corresponding values into the value table, thereby increasing coverage. Using this technique, AVPP can outperform state-of-the-art predictors. AVPP primarily improves efficiency by modifying the complete prediction process to increase coverage for load values, a specific type of instruction. While this thesis focuses on a load predictor, our approach could also be applied to predicting other instructions. [OAM18]

Second, and most closely related to this thesis, is Focused Value Prediction (FVP) by Bandishte et al. [Ban+20]. In this work, they aimed to make value prediction more efficient by explicitly targeting loads in long dependency chains to break them. To do so, they target loads that stall the retirement of the reorder buffer of an out-of-order processor. These loads are part of an especially long dependence chain, so that even out-of-order processing cannot hide the latency of the combined instruction. From

these instructions, they reconstruct the dependency chains by following the result and source registers. After connecting the instructions, they predict the last load before the stalling instruction, giving it more time to execute and avoid stalling. By targeting these instructions, they break up long dependence chains at crucial points, thereby increasing ILP. Using this method, they achieve performance comparable to state-of-the-art predictors while using one-eighth of the storage. [Ban+20]. FVP primarily focuses on an instruction's performance impact. It tries to predict loads, where a prediction can potentially save the most time. This is like the alternative criteria examined in Section 5.4. There, we used other thresholds for the two-level predictor to indicate potential for substantial time savings. We attempted to use the number of L1 cache misses and the maximum potential time saved as indicators of a valuable instruction. Both factors were too broad to yield any speedup; on the other hand, FVP can use this approach by employing a complex algorithm to reconstruct the longest dependency chains. Instead of focusing prediction on high-value loads, our design filters instructions based on the value-predictability of their first executions before they are even allowed to occupy space in the main prediction table. Consequently, these two approaches are complementary strategies for hardware optimization: one focuses on the instruction's performance impact, while the other optimizes for predictability to maximize storage efficiency. In summary, FVP and our two-level predictor make value prediction more efficient using different criteria. These criteria are not mutually exclusive and could be combined in future work.

7 Summary and Conclusions

Modern value prediction has become increasingly complex and resource-intensive. In particular, the prediction table is significantly larger than comparable components such as the L1 data cache or instruction cache. Unlike caches, not all values in a prediction table can be used directly, because they must first be trained. This makes prediction tables significantly less efficient, as only a portion of the space produces a speedup. For this reason, the speedup from a value predictor drops sharply as prediction tables become smaller.

We analyzed this by simulating the stride load-value predictor provided by Dr. David Schall in gem5 [Scha]. Here, we investigated multiple factors. At first, we examined the effect of the confidence threshold on the predictor's speedup. We observed that the predictor stabilized only for confidence thresholds above 50, and speedup was evident across all benchmarks. This is because, at lower thresholds, accuracy is not high enough, and we mispredict too often. For this reason, we used a threshold of at least 50 to ensure speedup across all benchmarks, and we applied it for all subsequent experiments. We decided against a higher confidence threshold because a higher threshold reduces coverage, as fewer instructions complete training. The problem with such high confidence is that training takes especially long and occupies more table space. The next step in our analysis was to investigate how many of all executed loads were predicted. We discovered that, on average, only 21% of all loads were ever predicted in the complete simulation. While the total number of predicted loads often fits in the table, it is often much larger. By trying to predict every load, we overload the table and push out loads that are viable for prediction. In a final step, we examined more closely how the lengthy training process affects the prediction table's behavior. For this, we analyzed a series of table snapshots that recorded how much table space was occupied by untrained, training, and trained loads. In these snapshots, we observed that only a small amount of table space was used for prediction, and most entries were occupied by training or untrained instructions. We also observed another pattern: many new loads were added to the table, pushing already trained loads out, which then had to start training from scratch. This shows that the table is used very inefficiently, as many entries are wasted on loads that will never contribute to a speedup. Combining these observations, we defined goals for a more efficient predictor design. The predictor should use the table more efficiently and avoid consuming most of the capacity for

non-predicting loads. It should also control the entry rate to avoid being overrun by rapid bursts of new loads. On top of this, it should maintain performance even with fewer resources and, therefore, avoid unnecessary hardware and latency overhead.

To achieve these goals, we introduce a filter positioned in front of the prediction table that stops instructions with a low likelihood of prediction from entering the main prediction table. We designed this filter as a second small table that already starts the first part of the training phase. If an instruction shows promising predictability in the filter and reaches a certain confidence value (the entry threshold), it is transferred to the main table, where the rest of the training is conducted, and active prediction occurs. If an instruction shows no predictability, it does not achieve the entry threshold and therefore never reaches the main prediction table, where it could push out trained instructions and reduce the efficiency of the prediction table. Fast bursts of unpredictable loads also do not impact the main prediction table because they never reach the entry threshold to the main table. This design avoids unnecessary hardware overhead because the filter intentionally uses a small table; it functions only during the first part of training and is not meant for long-term storage. It also does not introduce new latency to the prediction process as the filter only changes the training phase. Overall, the filter is not a new prediction algorithm but instead an extension to a conventional value predictor. It does not replace the underlying prediction mechanism; instead, it improves its practical applicability by making better use of the available hardware resources.

We implemented this filter design in the gem5 O3CPU model, on top of the stride predictor, and combined them into a two-level predictor[1].

To test this design, we used 13 server and SPEC benchmarks, along with a framework provided by Dr. David Schall [Schb]. As a first step, we ran simulations varying filter parameters, such as the entry threshold and the predictor table size, to identify the best combination. We found that for smaller tables, the predictor achieved the highest average speedup of 5.7% over a no-predictor simulation, with an entry threshold of 4 and 256 entries in the filter table. In contrast, for larger tables, the highest speedup of 6% was achieved with an entry threshold of 1 and 256 entries in the filter table. This means that, for small main tables, an entry threshold of 4 yields the best results, while for larger tables, a threshold of 1 performs better. Using these configurations, we tested both against the unmodified stride predictor and first examined how the main prediction table behaves. For this, we looked in exemplary detail at a 256-entry and an 8192-entry main prediction table. In all cases, we observed that the fraction of loads in the main table that are fully trained is substantially higher than that for the stride predictor. This difference was especially noticeable when we observed how

[1]The full implementation can be found in the gem5-lvp repository on GitHub [Lis].

the tables reacted to rapid bursts of many new instructions. While the stride predictor table lost many of the trained instructions, the burst was almost invisible in the main predictor table of the two-level predictors, as it was intercepted by the filter. Next, we compared the performance with the stride predictor. Here, we observed that for prediction tables smaller than 16K, the two-level predictor was significantly faster. The entry threshold 4 two-level predictor with a 256-entry filter and a main table size of 512 entries achieves performance comparable to that of the stride predictor using an 8192-entry prediction table. With an entry size of 19 bytes in the filter and prediction tables, the total table size drops from 156KB to 15KB, saving 154KB and requiring only 9.6% of the storage space. For larger sizes, both two-level predictors are slower than the standard stride predictor, as larger tables have fewer problems due to inefficiency, and the filter drastically restricts the two-level predictors' coverage. In a coverage analysis, we explicitly observe that at the maximum table size, the two-level predictor with an entry threshold of 4 has 5% lower coverage and with an entry threshold of 1, 2% lower coverage than the stride predictor. As a last experiment, we also analyzed how the predictor performed with other entry threshold parameters. We investigated the number of L1 misses and the maximum potential time saved through a prediction. Both indicate criticality rather than predictability and are often not directly tied to a specific load. Therefore, in both cases, we observed slowdowns and, in this work, did not investigate them further.

We examined other works that aim to improve the efficiency of value predictors. Specifically, we investigated FVP [Ban+20], which analyzes the critical path to focus value prediction on valuable loads. While it also works towards making value prediction more efficient, it approaches the topic from a different angle; therefore, it would be viable to combine both mechanisms in the future, which would be a further step towards a criticality-based design. Additionally, it would be interesting to implement the filter alongside a state-of-the-art predictor design, such as VTAGE [PS14], to determine whether it can achieve similar efficiency improvements.

We can conclude that we created a filter design that can be added onto existing value predictors to create two-level predictors. The filter selects instructions based on their predictability for the main value predictor, improving the predictor's efficiency. Using this filter, coupled with a stride predictor, we achieved results comparable to a standard value predictor while requiring only one-tenth as many table entries. This design is not yet a criticality-based value predictor, as it relies solely on instruction predictability for selection. Nevertheless, it represents a first step towards such a design by providing a storage-efficient framework that can incorporate additional selection criteria. Criticality information could be combined with predictability to prioritize instructions that are both likely to be predicted correctly and relevant to processor performance. Such an extension could focus on loads that lie on long dependence chains, cause significant

retirement delays, or provide substantial opportunities for latency reduction.

8 Future Work

Although the proposed two-level predictor already demonstrates more efficient storage usage, several opportunities for further improvement remain. The current design focuses on the predictability of load values but does not explicitly account for the criticality of individual instructions in the dependence chain. Extending the filter with this information could improve the selection of instructions that are forwarded to the main predictor.

In such an extension, criticality should not entirely replace predictability. Instead, it could be combined with the existing predictability criterion to distinguish between instructions that are merely predictable and those that are both predictable and relevant to processor performance. For example, the filter could forward an instruction to the main prediction table only if it meets a minimum predictability threshold and is associated with either a sufficiently long dependency chain or a significant retirement delay.

One possible implementation is therefore to add a second factor to the currently purely predictability-based filter, which estimates the performance impact of each instruction. Possible criteria for this include those explored in Section 5.4 (L1 misses, prediction-to-commit latency), which were not precise enough on their own but could be combined with more sophisticated criticality estimation techniques, such as those used in FVP [Ban+20]. This could be implemented as a combined threshold, where an instruction can favor one criterion more while still being used for prediction and remaining eligible. A further option would be to implement it as two separate criteria, both of which must be fully met for an instruction to access the main prediction table. Either implementation would allow us to focus prediction on the most critical values and use storage more efficiently.

Another interesting extension would be to apply the presented two-level design with state-of-the-art value predictors, such as the EVES [Sez18] or VTAGE [PS14] predictor. Both would need to be implemented in gem5 and combined with the two-level design. This would make it possible to study whether similar efficiency improvements can be achieved with modern value predictors without degrading their accuracy or coverage. When adding the filter to a more complex context-based predictor, it would be necessary to also use the complex algorithm for the first prediction table. Otherwise, only stride and constant predicted instructions could reach the main table, since the filter could

not predict more complex patterns. This is especially complicated for a predictor like VTAGE, which already relies on multiple tables to produce context-based predictions. It may be necessary to adapt both parts to combine them.

Further work should also include a more detailed analysis of the overhead introduced by the filter itself. Although the evaluation shows that, in the current implementation, a small filter table is sufficient, a future hardware-oriented study could examine latency, energy consumption, and implementation complexity in detail to determine how the two-level predictor performs under realistic constraints.

In summary, the proposed approach shows that stride value prediction can be made substantially more efficient by filtering for instructions likely to yield predictive values. Future work may build on this idea by integrating additional criticality information, assessing its impact on state-of-the-art predictors, and analyzing in detail the overhead incurred by this new approach.

Abbreviations

ILP Instruction-Level Parallelism

VTAGE Value Tagged Geometric History Length Predictor

EVES Enhanced VTAGE Enhanced Stride

FCM Finite Context Method

LRU Least Recently Used

FVP Focused Value Prediction

AVPP Address-first Value-next Predictor with Value Prefetchin

ITTAGE Indirect Target TAgged GEometric history length

PC Program Counter

List of Figures

2.1 Example structure of a value predictor table. Each entry stores the instruction's PC tag, confidence counter, last value, and other information necessary for predicting the next value and information for the replacement algorithm. 6
2.2 5-stage pipeline structure of the gem5 O3CPU model 8

3.1 Average stride predictor speedup of 13 different benchmarks compared to the no predictor baseline, plotted over the number of prediction table entries. 10
3.2 Stride predictor speedup across 13 different workloads compared to the no predictor baseline, plotted for confidence thresholds between 2 and 200. 11
3.3 Percentage of executed loads that were predicted at least once for 13 different workloads. 12
3.4 Count of executed loads and predicted loads for 13 different workloads. 12
3.5 Table-Snapshots over time for the benchmark nodeapp 14
3.6 Cutout of table snapshots for the nodeapp benchmark, comparing an 8K-entry table (a) with a 130K-entry table (b). The 8K table shows a sharp drop in active entries when many new loads arrive, while the larger table remains stable. 15

4.1 Graphic of the stride load value predictor implemented in gem5 20
4.2 Structure and data flow of the two-level load-value predictor implemented in gem5. The filter table performs the initial training phase and selectively inserts predictable loads into the main stride prediction table, which generates and verifies the value predictions. 23

5.1 2-table predictor speedup of 13 different benchmarks compared to the no predictor baseline, plotted over the number of prediction table entries in the filter table using an entry threshold of 4. 27
5.2 2-table predictor speedup with 8192-entry and 512-entry main tables, plotted over the entry threshold to the main table for a filter size of 256 entries. 28

5.3 Table snapshots over time for a 512-entry stride predictor using the nodeapp benchmark. The table shows a larger fraction of entries (almost all) in the untrained state compared to the two-level predictors in Figure 5.4 and Figure 5.5. 29
5.4 Table snapshots over time for the 256-entry two-level predictor with an entry threshold of 1 using the nodeapp benchmark. The table shows a larger fraction of entries in the training state (almost all) compared to the stride predictor in Figure 5.3, but significantly fewer in the active state than the 2-level predictor in Figure 5.5. 30
5.5 Table snapshots over time for the 256-entry two-level predictor with an entry threshold of 4 using the nodeapp benchmark. Compared with the stride predictor in Figure 5.3 and the two-level predictor with an entry threshold of 1 in Figure 5.4, the filter significantly increases the fraction of active loads. 31
5.6 Table snapshots over time for the 8192-entry two-level predictor with an entry threshold of 1 using the nodeapp benchmark. Compared with Figure 3.5, a higher fraction of entries is actively used for prediction, and fewer entries are completely untrained. 32
5.7 Table snapshots over time for the 8192-entry two-level predictor with an entry threshold of 4 using the nodeapp benchmark. Compared with Figure 3.5, a higher fraction of entries is actively used for prediction, while the strict filtering leaves the table partially empty. 33
5.8 Zoom-in on a rapid increase in newly encountered loads for the two-level predictor using the nodeapp benchmark, comparing an entry threshold of 4 (a) with an entry threshold of 1 (b). In both cases, the filter absorbs the rapid increase. 34
5.9 Average speedup of the stride predictor and the 2-level predictor with entry thresholds of 1 and 4, compared with the baseline (no predictor) as a function of the combined number of entries in the filter and the main prediction table. 36
5.10 Average coverage of the stride predictor and the 2-level predictor with entry thresholds of 1 and 4, compared with the baseline (no predictor), over the combined number of entries in the filter and the main prediction table. 37

List of Tables

5.1 Overview of the benchmarks used in the evaluation 25
5.2 Overview of the CPU configurations 25

Bibliography

[Abe+24] A. Abel, Y. Li, R. O'Grady, C. Kennelly, and D. Gove. "A Profiling-Based Benchmark Suite for Warehouse-Scale Computers." In: *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 2024, pp. 325–327. DOI: 10.1109/ISPASS61541.2024.00046.

[Ban+20] S. Bandishte, J. Gaur, Z. Sperber, L. Rappoport, A. Yoaz, and S. Subramoney. "Focused Value Prediction: Concepts, techniques and implementations presented in this paper are subject matter of pending patent applications, which have been filed by Intel Corporation." In: *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 2020, pp. 79–91. DOI: 10.1109/ISCA45697.2020.00018.

[Bla+06] S. M. Blackburn, R. Garner, C. Hoffmann, A. M. Khang, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, M. Hirzel, A. Hosking, M. Jump, H. Lee, J. E. B. Moss, A. Phansalkar, D. Stefanović, T. VanDrunen, D. von Dincklage, and B. Wiedermann. "The DaCapo benchmarks: java benchmarking development and analysis." In: *SIGPLAN Not.* 41.10 (Oct. 2006), pp. 169–190. ISSN: 0362-1340. DOI: 10.1145/1167515.1167488.

[GM98] F. Gabbay and A. Mendelson. "Using value prediction to increase the power of speculative execution hardware." In: *ACM Trans. Comput. Syst.* 16.3 (Aug. 1998), pp. 234–270. ISSN: 0734-2071. DOI: 10.1145/290409.290411.

[Gup+19] A. Gupta, P. Mor, H. Taneja, and B. Panda. "STEVES: Pushing the Limits of Value Predictors with Sliding FCM and EVES." In: *Championship Value Prediction (CVP)*. Available at the https://www.microarch.org/cvp1/cvp1online/contestants.html. 2019.

[HP17] J. L. Hennessy and D. A. Patterson. *Computer Architecture, Sixth Edition: A Quantitative Approach*. 6th. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2017. ISBN: 0128119055.

[KHI19] K. Koizumi, K. Hiraki, and M. Inaba. "H3VP: History Based Highly Reliable Hybrid Value Predictor." In: *Championship Value Prediction (CVP)*. https://microarch.org/cvp1/papers/Koizumi.pdf. 2019.

[Kim+25] J. Kim, J. Chuang, D. Genkin, and Y. Yarom. "FLOP: Breaking the Apple M3 CPU via False Load Output Predictions." In: *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 2595–2614. ISBN: 978-1-939133-52-6.

[Lis] J. Lison. *Development repository for Load Value Prediction (LVP) in gem5*. `https://github.com/g4mjak/gem5-lvp`. Accessed: 2026-08-17.

[Low+] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj, G. Black, G. Bloom, B. R. Bruce, D. R. Carvalho, J. Castrillon, L. Chen, N. Derumigny, S. Diestelhorst, W. Elsasser, C. Escuin, M. Fariborz, A. Farmahini-Farahani, P. Fotouhi, R. Gambord, J. Gandhi, D. Gope, T. Grass, A. Gutierrez, B. Hanindhito, A. Hansson, S. Haria, A. Harris, T. Hayes, A. Herrera, M. Horsnell, S. A. R. Jafri, R. Jagtap, H. Jang, R. Jeyapaul, T. M. Jones, M. Jung, S. Kannoth, H. Khaleghzadeh, Y. Kodama, T. Krishna, T. Marinelli, C. Menard, A. Mondelli, M. Moreto, T. Mück, O. Naji, K. Nathella, H. Nguyen, N. Nikoleris, L. E. Olson, M. Orr, B. Pham, P. Prieto, T. Reddy, A. Roelke, M. Samani, A. Sandberg, J. Setoain, B. Shingarov, M. D. Sinclair, T. Ta, R. Thakur, G. Travaglini, M. Upton, N. Vaish, I. Vougioukas, W. Wang, Z. Wang, N. Wehn, C. Weis, D. A. Wood, H. Yoon, and É. F. Zulian. *The gem5 Simulator: Version 20.0+*. `https://arxiv.org/abs/2007.03152`. Accessed: 2026-08-10.

[LS96] M. H. Lipasti and J. P. Shen. "Exceeding the dataflow limit via value prediction." In: *Proceedings of the 29th Annual ACM/IEEE International Symposium on Microarchitecture*. MICRO 29. Paris, France: IEEE Computer Society, 1996, pp. 226–237. ISBN: 0818676418.

[LWS96] M. H. Lipasti, C. B. Wilkerson, and J. P. Shen. "Value locality and load value prediction." In: *SIGOPS Oper. Syst. Rev.* 30.5 (Sept. 1996), pp. 138–147. ISSN: 0163-5980. DOI: 10.1145/248208.237173.

[OAM18] L. Orosa, R. Azevedo, and O. Mutlu. "AVPP: Address-first Value-next Predictor with Value Prefetching for Improving the Efficiency of Load Value Prediction." In: *ACM Trans. Archit. Code Optim.* 15.4 (Dec. 2018). ISSN: 1544-3566. DOI: 10.1145/3239567.

[Pro] gem5 Project. *gem5: Out of order CPU model*. `https://www.gem5.org/documentation/general_docs/cpu_models/O3CPU`. Accessed: 2026-08-09.

[Pro+19] A. Prokopec, A. Rosà, D. Leopoldseder, G. Duboscq, P. Tůma, M. Studener, L. Bulej, Y. Zheng, A. Villazón, D. Simon, T. Würthinger, and W. Binder. "Renaissance: benchmarking suite for parallel applications on the JVM." In:

Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation. PLDI 2019. Phoenix, AZ, USA: Association for Computing Machinery, 2019, pp. 31–47. ISBN: 9781450367127. DOI: 10.1145/3314221.3314637.

[PS14] A. Perais and A. Seznec. "Practical data value speculation for future high-end processors." In: *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. 2014, pp. 428–439. DOI: 10.1109/HPCA.2014.6835952.

[Sak+] C. Sakhuja, A. Subramanian, P. Joshi, and a. C. L. Akanksha Jain. *Combining Branch History and Value History For Improved Value Prediction*.

[Scha] D. Schall. *gem5 development repository for fetch-directed instruction prefetching*. https://github.com/dhschall/gem5-fdp/tree/value-pred. Accessed: 2026-08-10.

[Schb] D. Schall. *gem5 server benchmark suite*. https://github.com/dhschall/gem5-svr-bench/tree/multi-pred. Accessed: 2026-08-10.

[Sez18] A. Seznec. "Exploring value prediction with the EVES predictor." In: *CVP-1 2018 - 1st Championship Value Prediction*. Los Angeles, United States, June 2018, pp. 1–6.

[SM06] A. Seznec and P. Michaud. "A case for (partially) TAgged GEometric history length branch prediction." In: *Journal of Instruction-level Parallelism - JILP* 8 (Feb. 2006).

[SS97] Y. Sazeides and J. Smith. "The predictability of data values." In: *Proceedings of 30th Annual International Symposium on Microarchitecture*. 1997, pp. 248–258. DOI: 10.1109/MICRO.1997.645815.

[Sta17] Standard Performance Evaluation Corporation. *SPEC CPU 2017*. https://www.spec.org/cpu2017/. Accessed: 2026-08-14. 2017.